



On-Device Recommender Systems: A Comprehensive Survey

Hongzhi Yin¹ · Liang Qu² · Tong Chen¹ · Wei Yuan¹ · Ruiqi Zheng¹ · Jing Long¹ · Xin Xia¹ · Yuhui Shi³ · Chengqi Zhang⁴

Received: 12 May 2025 / Revised: 5 July 2025 / Accepted: 10 July 2025 / Published online: 13 November 2025
© The Author(s) 2025

Abstract

Recommender systems have been widely deployed in various real-world applications to help users identify content of interest from massive amounts of information. Traditional recommender systems work by collecting user-item interaction data in a cloud-based data center and training a centralized model to perform the recommendation service. However, such cloud-based recommender systems (CloudRSs) inevitably suffer from excessive resource consumption, response latency, as well as privacy and security risks concerning both data and models. Recently, driven by the advances in storage, communication, and computation capabilities of edge devices, there has been a shift of focus from CloudRSs to on-device recommender systems (DeviceRSs), which leverage the capabilities of edge devices to minimize centralized data storage requirements, reduce the response latency caused by communication overheads, and enhance user privacy and security by localizing data processing and model training. Despite the rapid rise of DeviceRSs, there is a clear absence of timely literature reviews that systematically introduce, categorize and contrast these methods. To bridge this gap, we aim to provide a comprehensive survey of DeviceRSs, covering three main aspects: (1) the deployment and inference of DeviceRSs, exploring how large recommendation models can be compressed and utilized within resource-constrained on-device environments; (2) the training and update of DeviceRSs, discussing how local data can be leveraged for model optimization on the device side; (3) the security and privacy of DeviceRSs, unveiling their potential vulnerability to malicious attacks and defensive strategies to safeguard these systems. Furthermore, we provide a fine-grained and systematic taxonomy of the methods involved in each aspect, followed by a discussion regarding challenges and future research directions. This is the first comprehensive survey on DeviceRSs that covers a spectrum of tasks to fit various needs. We believe this survey will help readers understand the current research status in this field, equip them with relevant technical foundations, and stimulate new research ideas for developing DeviceRSs.

Keywords Recommender systems · On-device learning · Model deployment and inference · Federated learning · Model attack and defense

Hongzhi Yin and Liang Qu have contributed equally to this work.

✉ Hongzhi Yin
h.yin1@uq.edu.au

Liang Qu
l.qu@ecu.edu.au

Tong Chen
tong.chen@uq.edu.au

Wei Yuan
w.yuan@uq.edu.au

Ruiqi Zheng
ruiqi.zheng@uq.net.au

Jing Long
jing.long@uq.edu.au

Xin Xia
x.xia@uq.edu.au

Yuhui Shi
shiyh@sustech.edu.cn

Chengqi Zhang
chengqi.zhang@polyu.edu.hk

¹ The University of Queensland, Brisbane, Australia

² Edith Cowan University, Perth, Australia

³ Southern University of Science and Technology, Shenzhen, China

⁴ The Hong Kong Polytechnic University, Hongkong, China

1 Introduction

Recommender systems (RSs), as an important technology to help online users efficiently pinpoint relevant information from the Web, have seen a strong uptake in many application sectors such as e-commerce [1], multimedia platforms [2], location-based services [3–7], etc. Typically, most existing RSs [1, 8, 9] are deployed on cloud server, where the recommendation model is trained and hosted in a centralized manner with data collected from users. The typical working mechanism of such cloud-based recommenders, termed *CloudRSs*, is depicted in Fig. 2a. However, given the intrinsic characteristics of this workflow, CloudRSs are inevitably subject to the following deficiencies: (1) **High Resource Consumption:** On the one hand, there is the substantial storage requirement for extensive data associated with users and items, including user-item interaction data, the features of users and items, and the parameters of the models, such as the weights of neural networks and user/item embeddings. On the other hand, the continuous updating and training of models to capture evolving user preferences also contribute substantially to resource consumption. (2) **Unwarranted Response Latency:** Another inherent limitation of the CloudRS architecture is the response latency between the cloud and user devices due to the communication overhead. For example, devices need to receive results from the CloudRS for display, and this response time can be impaired by limited Internet connection or high-traffic periods. Additionally, many models rely on historical instead of real-time user data for training and predictions, which often results in failing to capture real-time changes in user preferences. (3) **Security and Privacy:** Holding all user data in the data center could pose a risk to data security and privacy. In addition, with the release of data protection regulations such as GDPR¹ from Europe, CCPA² from the USA, and PIPL³ from China, there is a strong demand from users for enhancing data privacy protection. Furthermore, CloudRSs are vulnerable to attacks that can manipulate their outputs, leading to the generation of false or biased results.

Recent years have witnessed the rapid technical development in edge devices that has uplifted their storage, communication, and computational capacity. As such, a brand new recommendation paradigm, namely on-device recommender systems (DeviceRSs), is proposed to transit all or a bulk part of the computation and storage responsibilities from the cloud server to users' end-devices, such as cellphones, tablets, and smart home speakers. Given DeviceRSs' capability of addressing the aforementioned issues by putting

the model deployment and even training of recommendation models on the device side, there is also an emerging line of real-world applications such as Taobao's EdgeRec [10], Google's TFL Recommendation,⁴ and Kuaishou's short video recommendation on mobile devices [11]. Specifically, as systematically outlined in Fig. 1, existing DeviceRSs could be grouped into three broad categories, which are summarized in the following three paragraphs.

On-Device Deployment and Inference for DeviceRSs aim to deploy a lightweight recommendation model on resource-constrained devices, as shown in Fig. 2b. This allows for rapid on-device model inference, alleviating resource consumption and response latency issues. The primary technical challenge for this category of methods is how to retain the performance of the original model as much as possible while compressing it. The existing methods can be further categorized into several distinct types including Binary Code-based Methods [12, 13], Embedding Sparsification Methods [14, 15], Compositional Embedding Methods [7, 16–19], Variable Size Embedding Methods [20–22], and Sustainable Deployment methods [23].

Training and Update for DeviceRSs involves shifting the training process to the device side by leveraging locally stored data, as shown in Fig. 2c, thereby alleviating security and privacy concerns associated with the data uploading process. Additionally, local model updates can promptly capture changes in user preferences. However, the main challenge is that both standalone and collaborative training on devices face serious limitations: each device typically holds sparse, limited data and small models, while collaborative learning across devices further suffers from the non-IID distribution of user data, which hampers effective model aggregation. In order to address this issue, some methods in this field involve a central server that coordinates a group of devices for collaborative training between the server and devices (e.g., federated RSs [24–26]), and others adopt the P2P protocol to perform decentralized collaborative training among devices [27–29]. Recently, some research has focused on adapting the pre-trained large general models on the cloud server to the devices by fine-tuning using local data [30–32].

Security and Privacy for DeviceRSs aim to protect both users and on-device models from potential malicious attacks, as shown in Fig. 2d. The main challenge is twofold: (1) Privacy leakage through model updates. When local models contribute to a shared learning process, there is a risk of leaking sensitive user data. Even though direct user data might not be shared, sophisticated analysis of model updates can potentially reveal personal information. This is

¹ <https://gdpr-info.eu/>.

² <https://oag.ca.gov/privacy/ccpa>.

³ <https://personalinformationprotectionlaw.com>.

⁴ <https://www.tensorflow.org/lite/examples/recommendation/overview>.

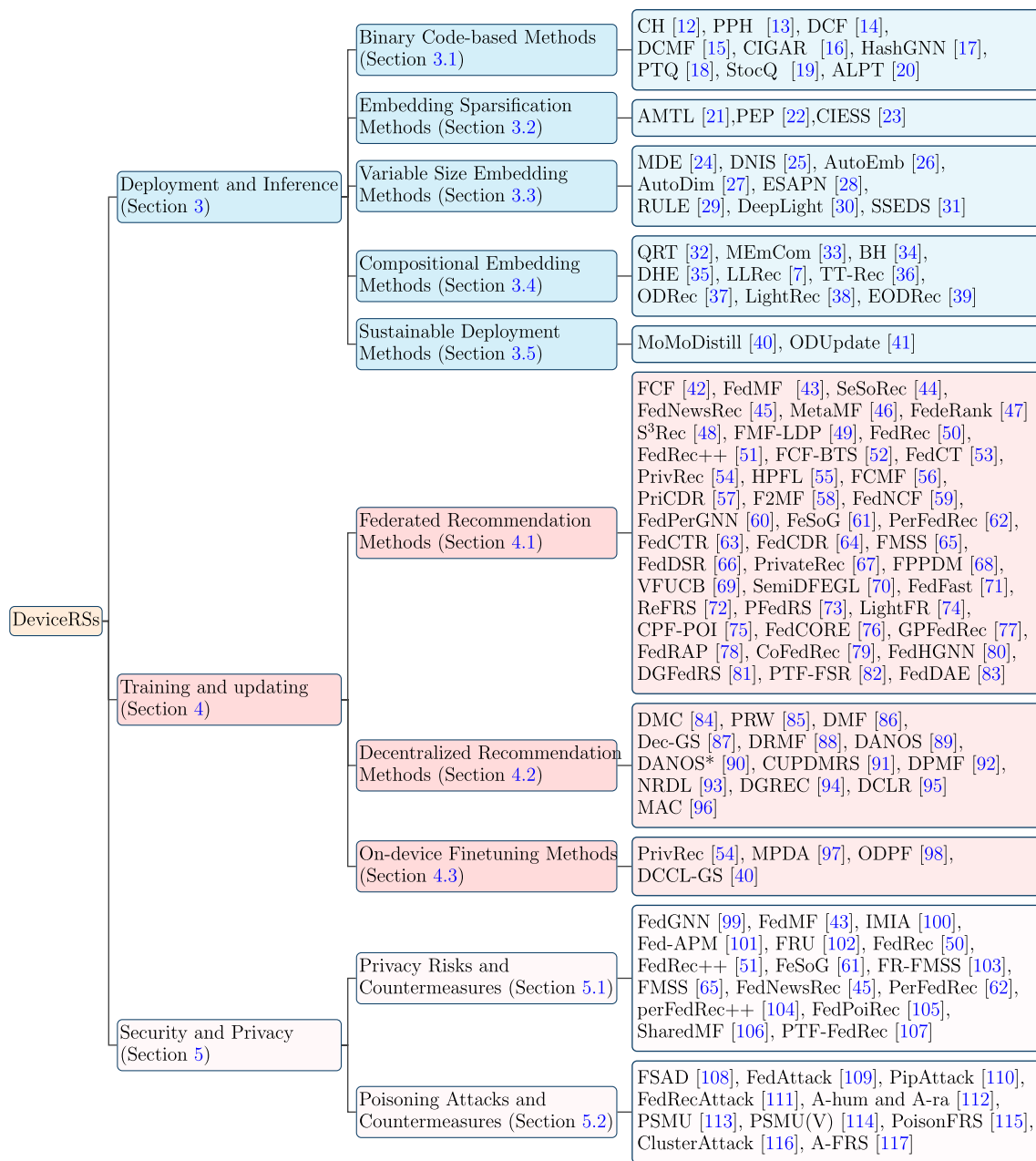


Fig. 1 Taxonomy of DeviceRSs

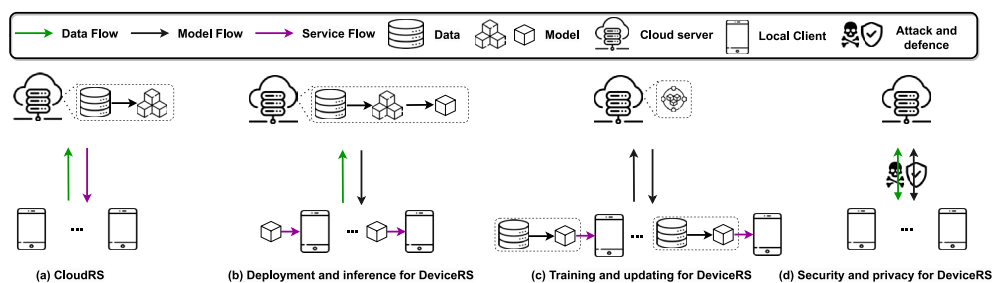


Fig. 2 The illustration of CloudRS and three types of DeviceRSs

especially true in scenarios where the model updates reflect unique user behaviors or preferences in the context of recommendation [33–36]. (2) Robustness against poisoning and adversarial attacks. Since the system relies on local data from user devices for training and updating models, there is a possibility that an attacker could manipulate this data or the model’s learning process. For instance, an attacker might introduce poisoned data or adversarial inputs into their local dataset, which could lead to the generation of inaccurate or biased recommendations [37–39].

There are a number of literature reviews on recommender systems and on-device learning. For example, Gao et al. [40] and Wu et al. [41] provide comprehensive reviews that focus on graph neural networks (GNNs) based recommender systems. Zheng et al. [42] and Tang et al. [43] introduce a range of work on recommender systems from an automated machine learning perspective. Yu et al. [44] build a systematic taxonomy on self-supervised recommendations. Li et al. [45] review methods for embedding compression in recommender systems. Zhang et al. [46] offer a comprehensive survey on Point-of-Interest (POI) recommender systems. Himeur et al. [47] and Rafi et al. [48] discuss the privacy preserving techniques in recommender systems. Wang et al. [49] present an overview of trustworthy recommender systems. On the other hand, Dhar et al. [50] summarize a set of works on on-device learning from the perspective of algorithms and learning theories, and Zhou et al. [51] present the on-device learning techniques from a software and hardware synergy perspective. However, these existing surveys treat recommender systems and the on-device learning in separate ways, so there is a timely demand for a comprehensive review that focuses on techniques in on-device recommender systems. The most similar works to ours are surveys on federated recommender systems [24–26, 52–55], but they only include limited works from the on-device training (i.e., federated learning) perspective, thereby missing the recent development of on-device deployment and attacking for recommendation. A detailed comparison of related surveys in terms of their coverage on deployment, training, and security aspects of DeviceRSs is provided in Table 1.

Due to the sheer volume of work in the field of DeviceRSs, it is impractical to cover every paper in our survey. Our criteria for selecting papers are based on those published in top-tier journals and conferences recognized in this field, including but not limited to TKDE (IEEE Transactions on Knowledge and Data Engineering), TOIS (ACM Transactions on Information Systems), and notable conferences like KDD (ACM SIGKDD Conference on Knowledge Discovery and Data Mining), SIGIR (ACM SIGIR Conference on Research and Development in Information Retrieval), WWW (The Web Conference), WSDM (ACM Conference on Web Search and Data Mining) and RecSys (ACM Conference on Recommender Systems). In

Table 1 Comparing existing surveys on their focus across deployment, training, and security in DeviceRSs

Survey	Deployment & inference	Training & update	Security & Privacy
Li et al. [45]	✓		
Sun et al. [24]		✓	
Yang et al. [25]		✓	
Alamgir et al. [26]		✓	
Asad et al. [52]		✓	
Harasic et al. [53]		✓	
Yang et al. [54]		✓	✓
Himeur et al. [47]			✓
Rafi et al. [48]			✓
Yuan et al. [55]		✓	✓
Ours	✓	✓	✓

addition to these peer-reviewed works, we also consider non-peer-reviewed valuable works, particularly those available on platforms like arXiv. However, our focus here is on those works that have gained significant recognition in the academic community and are deemed to be pioneering efforts in this field.

Overall, this paper aims to present an up-to-date and comprehensive review that covers a broader range of methods for DeviceRSs. The contributions of this survey are summarized as follows:

- **Comprehensive Review:** To our best knowledge, this is the first comprehensive survey of on-device recommender systems (DeviceRSs), offering an extensive overview of DeviceRSs technologies and their applications across various recommendation tasks. It includes an exploration of diverse approaches within DeviceRSs, highlighting their adaptability and functionality in different scenarios.
- **Systematic Taxonomy :** We propose a novel systematic taxonomy of the existing DeviceRS methods, focusing on three critical aspects: the deployment and inference for DeviceRSs, the training and updating for DeviceRSs, and the privacy and security for DeviceRSs. This taxonomy provides a structured insight into the state-of-the-art methods and their operational frameworks.
- **Challenges and Future Directions:** We dive into a detailed discussion regarding the potential challenges that DeviceRS methods currently face and propose future directions for research in this domain. This includes identifying gaps in the existing literature, suggesting improvements for current methodologies, and forecasting emerging trends that could influence the evolution of DeviceRSs.

The remaining sections of this survey are organized as follows: Section 2 introduces preliminary concepts for traditional CloudRSs and motivations of DeviceRSs. Sections 3, 4, and 5 dive into various aspects of DeviceRSs, covering deployment and inference, on-device training and updating, and privacy and security, respectively. Section 6 presents a set of commonly used evaluation metrics. Section 7 highlights potential challenges and future research directions, followed by a conclusion in Section 8.

2 Preliminary

Before we dive into the various types of DeviceRSs, we first provide relevant definitions of recommender systems and then discuss the motivations for shifting CloudRSs to DeviceRSs.

Recommender systems (RSs) have emerged as a popular research topic in both academia and industry over the past decade, aiming at assisting users in selecting items that align with their preferences from a vast array of options like news, videos, and products, thereby alleviating the issue of information overload. Generally, RSs perform recommendation services by collecting user data $\mathcal{D} = \{\mathcal{U}, \mathcal{I}, \mathcal{R}, \mathcal{X}\}$ to train a recommendation model $RS(\mathbf{P}, \mathbf{Q}, \Theta)$ for a given recommendation task T , which is defined as $RS(\mathbf{P}, \mathbf{Q}, \Theta) : \mathcal{D} = \{\mathcal{U}, \mathcal{I}, \mathcal{R}, \mathcal{X}\} \rightarrow T$, where $\mathcal{U}, \mathcal{I}, \mathcal{R}$, and $\mathcal{X}$ denote the user set, item set, user-item interactions, and features associated with users/items, respectively. $\mathbf{P}, \mathbf{Q}$, and Θ represent user representations, item representations, and model parameters, respectively. CloudRSs can be further classified into different categories based on the data, models, and recommendation tasks:

- Based on different data types, RSs can be categorized into collaborative filtering-based methods, which mainly utilize user-item interaction data $\mathcal{D} = \{\mathcal{U}, \mathcal{I}, \mathcal{R}\}$, and content-based methods (i.e., $\mathcal{D} = \{\mathcal{U}, \mathcal{I}, \mathcal{R}, \mathcal{X}\}$) that integrate side information associated users and items (e.g., user/item features) with user-item interaction data to improve recommendation performance.
- Depending on the recommendation model employed, RSs can be further divided. Matrix factorization-based meth-

ods (i.e., $RS(\mathbf{P}, \mathbf{Q})$) [56] decompose the user-item interaction matrix into two low-dimensional matrices for users and items. Each row (or column) vector in these matrices represents the user or item embeddings. With the rise of deep learning, many methods employ DNNs parameterized by parameters Θ to learn user/item embeddings to capture the nonlinear relationships between users and items, i.e., $RS(\mathbf{P}, \mathbf{Q}, \Theta)$ [2]. Additionally, since user-item interaction data can naturally be modeled as a user-item bipartite graph, some methods apply various graph neural network techniques to this bipartite graph to learn user/item embeddings [41].

- Based on different recommendation tasks, RSs can also be modeled for optimizing various objectives. For regression tasks, like rating prediction [57], the system predicts specific ratings that a user might give to items. For ranking tasks, such as top- k recommendation [58], it predicts the top k items most relevant to a given user. Lastly, for classification tasks like CTR (Click-Through Rate) prediction [8, 59], RSs predict whether a user will click on certain items.

3 Deployment and Inference for DeviceRS

In this section, we will introduce the deployment and inference methods for DeviceRSs, aimed at compressing large recommender system models to accommodate the memory limits of device-side platforms. The current trend involves transitioning models from well-resourced cloud environments to the more memory-restricted device side. In contrast to Natural Language Processing and Computer Vision, which utilize deep neural networks with substantial layers, recommender systems typically opt for shallower models. The majority of parameters in these systems are concentrated in the embedding tables. While some model compression methods [60–64] might lack explicit mention as on-device models in their original papers, they inherently possess the potential for deployment on devices. Therefore, we will also introduce a range of general recommendation model compression methods [65–67], highlighting their capabilities and potential benefits for deployment on devices. As depicted in Fig. 3, deployment and inference

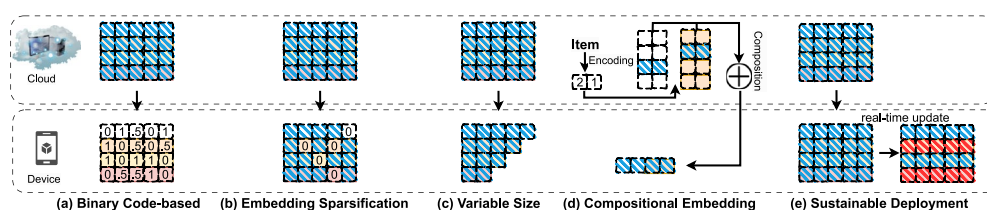


Fig. 3 The illustration of five types of methods within deployment and inference for DeviceRSs

methods for DeviceRSs primarily focus on embedding table compression. This typically includes: (1) Binary code-based methods involve converting embedding weights, typically stored in the format of FP32 (floating point 32-bit), to low-precision formats with fewer bits; (2) Embedding sparsification methods entail masking less important parts of embedding tables with zeros; (3) Variable size embedding methods aim to search the appropriate embedding length for different features; (4) Compositional embedding methods share weights among the embedding tables; (5) Sustainable deployment methods focus on achieving real-time on-device model updates post-deployment.

3.1 Binary Code-Based Embedding

Specifically, concerning the bit width of weights, Binary Code-based DeviceRSs can be categorized as binarization and quantization.

3.1.1 Binarization

It compresses an embedding weight from full-precision as binary code of just one bit, which presents two advantages: they demand less memory for storage, and the computation of similarity (inner product) becomes notably efficient through the employment of Hamming distance. This technique aligns with the broader trend of deploying recommender models on memory-restricted device-side platforms.

CH [68] and PPH [66] initially acquire compressed embeddings through binarization in two steps processes for Collaborative Filtering (CF) [69]. The procedure involves first learning embedding table with full precision solely without considering memory constraints. Subsequently, binarization is applied to these original embedding tables to obtain binary compressed embeddings. Nevertheless, such a compression step does not incorporated with the learning procedure and is unable to be learned jointly with the embedding table as a training objective. This limitation may result in substantial and irreparable errors. To address this issue and mitigate potential accuracy loss, subsequent efforts have shifted towards end-to-end frameworks. These methods aim to optimize compressed binary embeddings within the learning phase, enhancing the model's ability to maintain accuracy while incorporating binary constraints.

For direct optimization, binary embeddings are seamlessly integrated into the model parameters and directly optimized through the training loss. DCF [12] employs a binary embedding table $\mathbf{B} \in \{1, -1\}^{n \times d}$ alongside an original uncompressed embedding table $\mathbf{E} \in \mathbb{R}^{n \times d}$. The loss of $(\mathbf{B} - \mathbf{E})$ is incorporated into the optimization. Specifically, updates for $\mathbf{B}$ are performed through Discrete Coordinate Descent, while updates for $\mathbf{E}$ are facilitated by Singular Value Decomposition. DCMF [70] implements the same

optimization strategy and extends the task from rating prediction to top- k recommendation.

Differing from the direct optimization of binarization, indirect optimization involves generating the compressed embeddings from the original embeddings on the fly and optimizing them indirectly by optimizing the original full-precision embeddings. To address the challenge of zero gradients associated with the binary function (e.g., the sign function $\text{sign}(x)$), CIGAR [71] implements a scaled hyperbolic tangent function $\tanh(\alpha x)$, where $\lim_{\alpha \rightarrow \infty} \tanh(\alpha x) = \text{sign}(x)$. An alternative approach is the Straight-Through Estimator (SET) [72]. HashGNN [73] implements the STE variant of the $\text{sign}(x)$ function, optimizing $\mathbf{E}$ with the gradients of $\mathbf{B}$.

3.1.2 Quantization

To strike a balance between prediction accuracy and memory cost, quantization employs a multi-bit integer to represent each weight, mitigating accuracy drop associated with strict binarization. This process entails mapping an embedding weight consisting of 32-bit full-precision to a value within a set of quantized values.

In the context of post-training quantization (PTQ) [74], two quantization approaches are implemented. The first approach is uniform quantization, which involves upholding a range for each embedding to be quantized. The optimal range is determined through a greedy search. The second approach is non-uniform quantization, which involves grouping similar embeddings and employing clustering for each group to generate a codebook. The weights within each group are subsequently connected to the index value of the corresponding codebook. Recent works such as StocQ [59] and ALPT [75] have introduced approaches to learning quantized weights from scratch. In contrast to directly quantizing full-precision weights in the forward manner and updating them with gradients estimated by STE, StocQ, and ALPT store weights in integer format during training to compress model memory size. Then de-quantized weights are taken as the input of the model, and after backward propagation, the model re-quantizes the weights back into integers. Given that the input of recommendation systems (e.g., one-hot vectors) is highly sparse, only a small portion of the embeddings will then be de-quantized by the model into floating-point values.

3.2 Embedding Sparsification

Embedding sparsification methods, in contrast to binary code-based embedding methods that convert embedding weights to low-precision formats with fewer bits, identify less crucial embedding weights. Subsequently, these weights are masked with zeros using a sparsification technique.

ATML [65] suggests using an Adaptively-Masked Twins-based layer to reduce useless dimensions in the original embedding table through the mask vector. It incorporates feature frequency knowledge, employing l-AML and h-AML branches to handle low- and high-frequency instances, preventing dominance of parameters by high-frequency samples.

Learnable thresholds play an important role in learning the significance of parameters in the embedding matrix. Drawing inspiration from Soft Threshold Reparameterization, **PEP** [14] dynamically prunes by adaptively reparameterizing the embedding matrix using $\hat{\mathbf{E}} = \mathcal{S}(\mathbf{E}, s) = \text{sign}(\mathbf{E})\text{ReLU}(\text{abs}(\mathbf{E}) - (1 + e^{-s})^{-1})$, where $\hat{\mathbf{E}}$ and $\mathbf{E}$ denote the re-parameterized and original embedding matrices, respectively. The learnable threshold(s) s are updated via gradient descent, and $\text{sign}(\cdot)$ represents the sign function. Furthermore, OptEmbed [76] not only employs learnable pruning thresholds for redundant embeddings but also implements a field-wise dimension mask for better performance.

Reinforcement learning offers another avenue for embedding sparsification. Unlike other RL-based methods with discrete choices, CIESS [15] adopts an actor-critic paradigm, including the twin delayed deterministic policy gradient, to navigate a continuous search space, allowing flexible and arbitrary adjustment of embedding sizes.

3.3 Variable Size Embedding

Similar to embedding sparsification that assigns 0 to a certain embedding weight to shrink memory consumption, variable size embedding DeviceRSs methods assign different sizes of embedding dimensions to each feature (field). It can not only save memory but also lead to better performance. Given the long-tail distribution observed in recommendation tasks [77], assigning identical dimensions to both head and tail features is not rational. High-frequency features, typically following a long-tail distribution, require more weight to store the extensive information. Conversely, representing low-frequency features with excessive parameters could lead to overfitting because of limited training data.

One possible search strategy is based on human rules, where MDE [78] proposes a mixed-dimension embedding layer that dynamically learns the dimension of the embedding according to the popularity of features. Specifically, the MD layer is composed of F blocks, each matching to F feature fields in the context of the CTR task. Every block instance is reshaped into two matrices: the projection matrix and the embedding matrix. MDE initially establishes a block-level probability vector, and then the mixed embedding dimensions based on the popularity and the two matrices.

Motivated by the success of Neural Architecture Search (NAS) [79], a research line treats embedding optimization as a hyper-parameter optimization problem. This involves searching the needed embedding dimensions from a human-defined search space. NIS [67] learns vocabulary sizes and embedding dimensions for unique features. Building on this, Differential Neural Input Search (DNIS) [80] enhances efficiency by introducing a differential framework, enabling continuous dimension selection instead of pre-defined discrete dimension sets. AutoEmb [81], an AutoML-based framework for sequential recommendations, dynamically selects embedding dimensions based on changing popularity for users and items in ranking prediction tasks. AutoDim [82] broadens this scope to recommendation tasks with multiple feature fields by assigning dimensions for different feature fields. ESAPN [20] dynamically searches embedding dimensions with reinforcement learning, employing a hard selection strategy, and diverging from the soft selection approach utilized by AutoEmb to effectively reduce storage space.

This problem can also be formulated as embedding pruning optimization, where pruning is applied to the entire embedding matrix using various strategies to automatically achieve variable-sized embeddings. Deeplight [83] addresses high-latency concerns in CTR prediction by proposing pruning for both embedding and DNN layers. This involves pruning weight matrices in the DNN component to eliminate connections, resulting in a sparse DNN with reduced computational complexity and accelerated training. Additionally, field pair interaction matrices are pruned for field pair selection. RULE [21] learns the full embedding table and searches for elastic embeddings for multiple groups of items using evolutionary search. As a once-for-all framework, it accommodates on-device recommenders with varying memory restrictions without training from scratch. SSEDs [84] presents embedding pruning in a single-shot manner. Initially, a recommendation model is pre-trained with the same embedding dimensions. Following this, a proposed criterion is employed, measuring the importance of each embedding dimension in a single forward-backward pass. This process results in salience scores assigned to each dimension to fit the memory budget.

3.4 Compositional Embedding

Binary code-based methods decrease the bit count of embedding weight, while embedding sparsification and variable size embedding techniques shorten the embedding length. In contrast, compositional techniques share embedding weights among embedding tables, effectively reducing the overall parameter count.

We extend the concept of compositional embedding and utilize a weight-sharing framework proposed by current

methods [45]. Concretely, the generation of embedding is defined as $\mathbf{e} = \sum / \bigcup_{i=1}^s \mathbf{I}^i \times \mathbf{T}^i$. $\sum / \bigcup$ represents summation or concatenation, $\mathbf{I}^i$ are meta-embeddings to determine shared vectors in $\mathbf{T}$, and $\mathbf{T}^i$ are meta-tables composed of shared vectors.

One kind of approach is hashing methods, which learn the meta-embeddings by employing hash functions to the original feature ID. Each feature is represented by an embedding $\mathbf{e} = \mathbf{I} \times \mathbf{T}$, where $\mathbf{T} \in \mathbb{R}^{m \times d}$, $m < n$, $\mathbf{I} = \mathbb{I}(ID \% m) \in \{0, 1\}^m$, and $\mathbb{I}$ is the one-hot encoding function. To reduce the collisions raised by mapping many features to the same embedding, multiple hash functions and index vectors with several hash function candidates are utilized for feature ID. For instance, QRT [16] employs two meta-tables and utilizes remainder model and quotient model to learn dual index vectors, respectively. Following this, MEmCom [85] implement two meta-tables ($\mathbf{T}^1 \in \mathbb{R}^{m \times d}$, $\mathbf{T}^2 \in \mathbb{R}^{n \times 1}$) and dual index vectors ($\mathbf{I}^1 = \mathbb{I}(ID \% m)$, $\mathbf{I}^2 = \mathbb{I}(ID)$), multiplying two meta-embeddings to obtain the final embedding to better distinguish multiple features. Furthermore, BH [86] clusters the full-precision feature ID into s partitions and assigns them into $(ID_1, \dots, ID_s)$. Every subgroup correlates to an index vector $\mathbf{I}^i = \mathbb{I}(ID_i)$ and obtains a meta-embedding for sharing among all $\mathbf{T}^i$.

Another research line is vector quantization, which learns the index vectors by the nearest-neighbor search to capture the similarity between features themselves. It's worth noting that in recent literature on vector quantization, the terms codeword and codebook are commonly used to refer to what we denote as meta-embedding and meta-table. For the original embedding $\mathbf{e}$, vector quantization obtains the index vector by $\mathbf{I} = \mathbb{I}(\arg \max_k \text{sim}(\mathbf{e}, \mathbf{T}_k)) \in \{0, 1\}^m$, where $\mathbf{T}_k$ is the meta-embedding and $\text{sim}(\cdot)$ is the function to calculate similarity. For instance, LightRec [17] uses vector quantization to compress the item embedding tables and utilizes a pre-trained teacher model for efficiency. Furthermore, EODRec [19] proposes a knowledge distillation paradigm with bidirectional self-supervised learning and saves meta-tables on local device for fast local inference.

As for decomposition techniques, they implement soft selection through aggregating across all meta embeddings within a meta-table $\mathbf{T} \in \mathbb{R}^{m \times d}$ with an index vector $\mathbf{I} \in \mathbb{R}^m$ containing real values. Considering the extensive representational capacity of the index vectors, only one meta-table is satisfactory for addressing feature collisions. Every feature is represented by a unique index vector in $\mathbf{I}_M \in \mathbb{R}^{n \times m}$ when expressing the decomposition as $\mathbf{E} = \mathbf{I}_M \times \mathbf{T}$. DHE [22] adopts a hash encoder $\mathcal{H} : \mathbb{N} \rightarrow \mathbb{R}^m$ for better optimization and storage of $\mathbf{I}_M$. To overcome the limitations of deterministic index vectors that cannot be optimized, DHE takes a step further by decomposing the meta-table $\mathbf{T}$ into a DNN to enhance the expressive capacity of the meta-table.

Furthermore, tensor train decomposition (TTD) is employed for the decomposition of the embedding tables. The embedding table $\mathbf{E} \in \mathbb{R}^{n \times d}$ is firstly reshaped into $\mathcal{E} \in \mathbb{R}^{(n_1 d_1) \times (n_2 d_2) \times \dots \times (n_s d_s)}$, where $n = \prod_{i=1}^s n_i$ and $d = \prod_{i=1}^s d_i$, then, $\mathcal{E}$ is decomposed into $\mathcal{E} = \mathcal{G}_1 \times \mathcal{G}_2 \times \dots \times \mathcal{G}_s$, where $\mathcal{G}_i \in \mathbb{R}^{r_{i-1} \times n_i d_i \times r_i}$ is called TT-core and r_i is called TT-rank. TT-Rec [87] stands out as the pioneer in applying Tensor-Train Decomposition (TTD) to recommendation tasks. This implementation involves optimized kernels specifically designed for TTD on embedding tables. On a similar note, LLRec [7] utilizes TTD on embedding tables with a focus on preserving prediction accuracy through knowledge distillation. Facing the challenge of balancing the trade-off between under-expressive embedding approximations due to a small TT-rank in TTD and the sacrifice of model efficiency with a larger TT-rank, ODRc [18] introduces semi-tensor product-based tensor train decomposition (STTD) as an enhancement to TTD for extremely compressing the embedding table. This adjustment enables TTD to perform tensor multiplication across varying ranks, effectively addressing concerns regarding efficiency and effectiveness.

3.5 Sustainable Deployment Methods

The widespread adoption of compression techniques has offered solutions for deploying recommendation models on devices. However, a notable challenge has emerged concerning the sustainable deployment of on-device recommendations. Here, sustainable refers to the ability to keep models updated with evolving user interests under limited device resources. With millions of user interactions generated incessantly, this task becomes particularly demanding. However, device models can not achieve real-time updates for the lack of computing ability and storage space. One strategy is making cloud-device communication efficient where DeviceRSs can still benefit from the cloud. MoMo-Distill [31] first studies Device-Cloud Collaboration Learning framework to benefit both sides jointly. MoMoDistill reduces the parameter size of the cloud model to fit devices and then enhances the cloud modeling via the knowledge distillation sourced from the personalized device models. ODUpdate [23] proposes to compress model updates over time by encoding the model update into the compositional codes of items. Besides, they also designed two updating methods based on stack and queue, achieving different kinds of minimal communication costs.

4 Training and Update for DeviceRSs

In this section, we will introduce the training and update methods for DeviceRSs, which means that a part or all of the model training and updating process is put on the device

side, thereby alleviating security and privacy concerns and enhancing the ability to capture changes in user preferences. Specifically, due to the inherent data sparsity of user-item interactions, the training data on each device is usually limited, making it challenging to achieve good performance through local data training alone. Consequently, methods in this field typically involve a central server that coordinates a group of devices in different manners including (1) Federated recommendation methods that typically rely on a central server to coordinate model training via device-to-server communications; (2) Decentralized recommendation methods that typically perform device-to-device collaborative training among neighbors assigned from the server; (3) On-device finetuning methods that typically leverage local data to refine the pre-trained models from the server side. We elaborate these three types of methods separately in the following.

4.1 Federated Recommender Systems

As mentioned before, traditional cloud-based recommender systems raise privacy concerns due to the fact that they collect all user data on a central server. Recently, inspired by the fact that federated learning has achieved promising performance in privacy-preserving machine learning, many research works have attempted to incorporate federated learning with recommender systems, termed federated recommender systems (FedRSs), to address privacy concerns. In the subsequent part, we will first provide a problem formulation of FedRSs, and then introduce the typical pipelines of FedRSs. Finally, we will categorize FedRSs based on the different methods adopted in the FedRS pipeline. Table 2 chronologically summarizes major FedRS methods and their addressed tasks, such as rating prediction [89, 93, 97, 106], Top- k recommendation [88, 91, 117, 118], and CTR prediction [109, 115]. Additionally, some methods are designed to integrate specified side information in order to complete specific recommendation tasks. These include social recommendations that incorporate users' social information [90, 95], sequential recommendations based on user behavior information [111, 116], and news recommendations integrating item attribute information [92, 113]. Furthermore, FedRSs can be categorized based on the types of participants. Cross-client FedRSs [88, 89, 111] involve individual users or clients as training participants, each holding only their own user-item interaction data. This setting aligns with the definition of Horizontal Federated Recommendation in the federated learning literature [25], where participants share the same feature space (e.g., items) but differ in user identities. This is also the most common setup for on-device recommendation, where local models are trained directly on users' devices based on their own interaction history. In

contrast, cross-platform FedRSs [90, 95, 109] involve multiple service providers (e.g., companies or platforms), each possessing a complete set of user-item interaction data under their respective ecosystem. This corresponds to Vertical Federated Recommendation, where different participants may share overlapping user bases but hold different feature sets or item spaces. This category often includes cross-domain recommendation scenarios [100, 103, 114], where the data originates from different domains or systems.

4.1.1 Problem Formulation of FedRSs

Let $\mathcal{U}$ and $\mathcal{I}$ denote a set of users and items, respectively. Each user $u \in \mathcal{U}$ holds a local dataset $\mathcal{D}_u = \{\mathcal{I}_u, \mathbf{y}_u\}$, where $\mathcal{I}_u \subseteq \mathcal{I}$ denotes a set of items interacted with user u , and $\mathbf{y}_u \in \mathbb{R}^{|\mathcal{I}|}$ or $\mathbf{y}_u \in \{0, 1\}^{|\mathcal{I}|}$ represent explicit feedback (e.g., ratings) or implicit feedbacks (e.g., clicks and check-ins) of users, respectively. In addition, each device maintains a local model $f_u(\mathbf{p}_u, \mathbf{Q}_u, \Theta_u)$ that typically consists of parameters including d -dimensional user embedding $\mathbf{p}_u \in \mathbb{R}^d$, item embedding table $\mathbf{Q}_u \in \mathbb{R}^{|\mathcal{I}| \times d}$, and model parameters Θ_u . The server correspondingly maintains a global model $f_s(\mathbf{Q}_s, \Theta_s)$ that typically does not include sensitive user embedding $\mathbf{p}_u$. Finally, the goal of FedRSs is to learn the optimal global parameters $(\mathbf{Q}_s^*, \Theta_s^*)$ by coordinating a set of devices to perform local training.

4.1.2 Typical Pipeline of FedRSs

As shown in Fig. 4, the pipeline of FedRSs typically consists of four stages including: (1) **Client Selection**: for each training round, the server first selects a set of devices to participate in this round of training based on the predefined client selection strategy, and then distributes the global parameters of the server to these clients. (2) **Local Training**: the clients selected to participate in training perform training on a local model based on a local dataset. (3) **Model Upload**: after local training, the device uploads parameters or corresponding gradients to the server. (4) **Global Aggregation**: the server employs the model aggregation strategy to obtain a global model based on uploaded parameters/gradients. After that, the updated global parameters are redistributed to each client. It is worth noting that there is no actual data sharing between devices and the server throughout the training pipeline of FedRSs for privacy concerns. Below, we will introduce the strategies used for the different methods in each stage, summarized in Table 3.

4.1.3 Client Selection

In each training round, the server first selects a set of clients to participate in the current training round. It then distributes the latest global parameters to these clients.

Table 2 Summary of main FedRSs with their addressed tasks and types of participants

Method	Year	Venue	Task	Type
FCF [88]	2019	ArXiv	Top- <i>k</i> recommendation	Cross-client
FedMF [89]	2020	IEEE Intelligent Systems	Rating prediction	Cross-client
SeSoRec [90]	2020	ECAI 2020	Rating prediction	Cross-platform
FedFast [91]	2020	KDD 2020	Top- <i>k</i> recommendation	Cross-client
FedNewsRec [92]	2020	EMNLP 2020	Top- <i>k</i> recommendation	Cross-client
MetaMF [93]	2020	SIGIR 2020	Rating prediction	Cross-client
FedeRank [94]	2021	ECIR 2021	Top- <i>k</i> recommendation	Cross-client
S ³ Rec [95]	2021	NeurIPS 2021	Rating prediction	Cross-platform
FMF-LDP [96]	2021	RecSys 2021	Top- <i>k</i> recommendation	Cross-client
FedRS [97]	2021	IEEE Intelligent Systems	Rating prediction	Cross-client
FedRS++ [98]	2021	AAAI 2021	Rating prediction	Cross-client
FCF-BTS [99]	2021	RecSys 2021	Top- <i>k</i> recommendation	Cross-client
FedCT [100]	2021	SIGIR 2021	Top- <i>k</i> (cross-domain)	Cross-platform
PrivRec [30]	2021	VLDB Journal	Top- <i>k</i> recommendation	Cross-platform
HPFL [101]	2021	WWW 2021	Rating prediction	Cross-platform
FCMF [102]	2021	Knowledge-Based Systems	Rating prediction	Cross-platform
PriCDR [103]	2022	WWW 2022	Top- <i>k</i> (cross-domain)	Cross-platform
F2MF [104]	2022	RecSys 2022	Top- <i>k</i> recommendation	Cross-client
FedNCF [105]	2022	Knowledge-Based Systems	Top- <i>k</i> recommendation	Cross-client
FedPerGNN [106]	2022	Nature Communications	Rating prediction	Cross-client
FeSoG [107]	2022	TIST	Rating prediction	Cross-client
PerFedRS [108]	2022	CIKM 2022	Top- <i>k</i> recommendation	Cross-client
FedCTR [109]	2022	TIST	CTR Prediction	Cross-platform
FedCDR [110]	2022	CIKM 2022	Rating prediction	Cross-platform
FMSS [111]	2022	TOIS	Rating prediction, Top- <i>k</i> , Sequential recommendation	Cross-client
FedDSR [112]	2023	TKDE	Sequential recommendation	Cross-client
PrivateRec [113]	2023	KDD 2023	Top- <i>k</i> recommendation	Cross-client
FPPDM [114]	2023	IJCAI 2023	Top- <i>k</i> (cross-domain)	Cross-platform
VFUCB [115]	2023	KDD 2023	CTR Prediction	Cross-platform
SemiDFEGL [28]	2023	WWW 2023	Top- <i>k</i> recommendation	Cross-client
ReFRS [116]	2023	TOIS	Sequential recommendation	Cross-client
PFedRS [117]	2023	IJCAI 2023	Top- <i>k</i> recommendation	Cross-client
LightFR [118]	2023	TOIS	Top- <i>k</i> recommendation	Cross-client
CPF-POI [119]	2024	TKDE	POI recommendation	Cross-client
FedCORE [120]	2024	TKDE	Rating prediction	Cross-platform
GPFedRec [121]	2024	KDD 2024	Top- <i>k</i> recommendation	Cross-client
FedRAP [122]	2024	ICLR 2024	Top- <i>k</i> recommendation	Cross-client
CoFedRec [123]	2024	WWW 2024	Top- <i>k</i> recommendation	Cross-client
FedHGNN [124]	2024	WWW 2024	Top- <i>k</i> recommendation	Cross-client
DGFedRS [125]	2025	TOIS	Top- <i>k</i> recommendation	Cross-client
PTF-FSR [126]	2025	TOIS	Top- <i>k</i> recommendation	Cross-client
FedDAE [127]	2025	AAAI 2025	Top- <i>k</i> recommendation	Cross-client

However, the data distribution of each client is usually non-independent and identically distributed (non-iid). Therefore, the method of selecting clients for each training round is crucial, as it can significantly affect the model's performance. In this section, we refer to this process as the

client selection [130], and introduce various client selection strategies below.

- **Random Selection** [30, 92–94, 100, 104–107, 110–113, 117]: Most existing FedRS methods adopt a ran-

Fig. 4 The pipeline of FedRSs typically consists of four steps including client selection, local training, model upload, and global aggregation

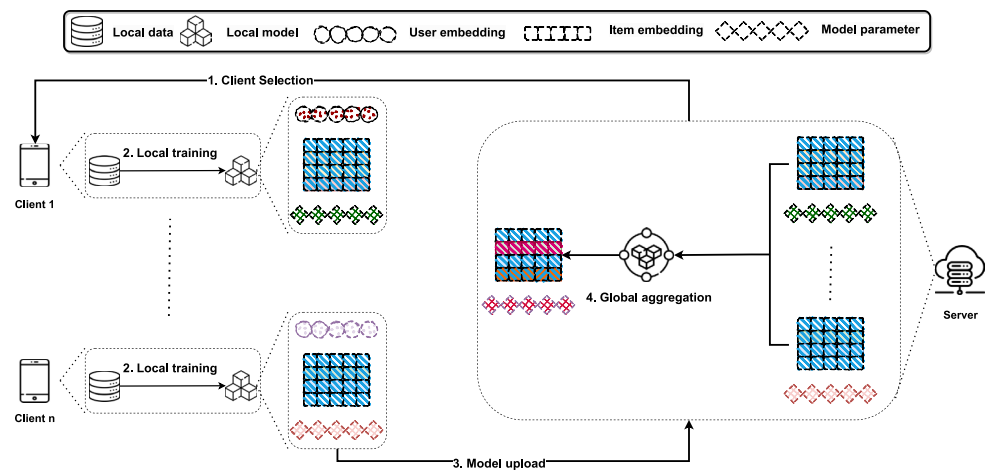


Table 3 Summary of the strategies used in the different stages of FedRSs

FedRS pipeline	Strategy	References
Client selection	Random selection	[30, 92–94, 100, 104–107, 110–113, 117]
	Full selection	[88–90, 95–99, 101–103, 109, 114, 115]
	Clustering-based selection	[28, 91, 108, 116]
Local training	MF-FedRSs	[88–90, 94–98, 101, 102, 104, 118]
	DNN-FedRSs	[30, 91–93, 100, 103, 105, 109–111, 113, 116, 117]
	GNN-FedRSs	[28, 106–108, 114]
	RL-FedRS	[99, 112, 115]
	Others	[99, 101, 114, 118]
Model upload	Homomorphic encryption	[89, 95, 102, 106, 116]
	Secret sharing	[111, 128]
	Secure multiparty computation	[90, 95]
	(Local) Differential privacy	[28, 30, 92, 96, 102–104, 106, 107, 109, 113, 115–117]
	Fake items	[28, 97, 98, 106, 107, 111]
	Mask matrix	[105]
	Others	[99, 101, 114, 118]
Global aggregation	Gradient descent	[88–90, 93–95, 97–99, 102, 109]
	FedAvg	[28, 104, 106–108, 110, 111, 129]
	Average aggregation	[30, 92, 96, 100, 114, 117]
	Others	[91, 101, 105, 112, 113, 118]

dom selection strategy, where a set of clients is selected randomly in each training round. Such client selection methods are generally applicable when there is a large number of clients, such as the cross-client-based FedRSs methods.

- Full Selection** [88, 89, 96–99, 101]: Methods that select all clients for each training round are termed full selection. Although this type of methods require high communication costs between the server and clients, making them unsuitable for scenarios with a very large number of clients, they are generally applicable to cross-platform FedRS methods [90, 95, 102, 103, 109, 114, 115] where each recommendation service platform serves as a client.
- Clustering-based Selection** [28, 91, 108, 116]: There are also clustering-based methods for client selection. For example, ReFRS [116] calculates the semantic similarity of clients based on the model parameters they upload and employs dynamic clustering techniques to generate discriminative user clusters for later global aggregation. SemiDFEGL [28] utilizes fuzzy c-means clustering to group the ego-graph embeddings uploaded by users into different clusters. Subsequently, clients are sampled individually within each group for global aggregation. PerFedRS [108] clusters users into a set of groups based on uploaded user embeddings, and within each group, clients are randomly selected in proportion to the cluster size. FedFast [91] introduces an active sampling strategy

for client selection strategy, namely ActvSAMP, which utilizes k-means to cluster users into a set of groups based on their metadata or user embeddings and then randomly sample one client from each group to participate in federated training.

4.1.4 Local Training

After the client selection stage in each training round, each selected client performs local model training using the local dataset and global parameters received from the server. Specifically, each local dataset only contains the user's own interaction data between the user and items. In addition, the local parameters typically consist of three components including the user embedding, the item embedding table, and the model's parameters. In each training round, the global parameters received from the server are used to initialize the local model parameters. In this way, the client can perform standard model training locally. Below, we will introduce various recommendation methods utilized for local training, including matrix factorization-based methods (MF-FedRSs), deep neural networks-based methods (DNN-FedRSs), graph neural networks-based methods (GNN-FedRSs), and reinforcement learning-based methods (RL-FedRSs).

- MF-FedRSs** [88, 89, 94, 96–99, 102, 104] typically utilize the user embedding matrix $\mathbf{P} \in \mathbb{R}^{|\mathcal{U}| \times d}$ and the item embedding $\mathbf{Q} \in \mathbb{R}^{|\mathcal{I}| \times d}$ to represent users and items, respectively. Then, the preference $\mathbf{R}$ of users $\mathcal{U}$ for items $\mathcal{I}$ can be predicted by $\mathbf{R} = \mathbf{P}\mathbf{Q}^T$. In the context of FedRSs, each user u only maintain their own user embedding $\mathbf{p}_u$ and item embedding table $\mathbf{Q}_u$ locally. Formally, each client can calculate the local loss $\mathcal{L}_u$ based on the local dataset $\mathcal{D}_u$. The choice of loss function depends on the specific recommendation task, such as the Bayesian Personalized Ranking (BPR) in the Top- k recommendation [131]. In this way, the local user/item embeddings can be updated by the gradient descent locally. Furthermore, LightFR [118] argues that the above real-valued MF methods could lead to privacy concerns and require substantial communication costs and storage space in the server-device communication process. To address this issue, it proposes a method based on a binary matrix by the learning-to-hash technique. Additionally, a corresponding discrete optimization method is introduced during the local training process to optimize the local binary matrix. HPFL [101] proposes a general user model (GUM) to perform local training, including public and private components. The former is used to learn non-sensitive information, such as item attributes, via multi-hot vectors, while the latter is employed to learn private user-item interaction information via MF. Finally, the knowledge matrix fuses the public item attribute
- representations with the user/item private embeddings, forming the final user/item representations. S³Rec [95] and SeSoRec [90] leverage additional social information from another platform to alleviate the data sparsity issue in user-item rating matrix. They adopt Soreg [132] to learn user/item embeddings based on both the user-item rating matrix and the user-user social matrix.
- DNN-FedRSs** generally integrate additional deep neural network components with MF to capture the complex non-linear relationships between users and items. For example, some methods [93, 105, 117] utilize Generalized Matrix Factorisation (GMF) [133] that employs multi-layer perceptron (MLP) as the score function to calculate the similarity score s_{ij} between user embedding $\mathbf{p}_i$ and item embedding $\mathbf{q}_j$. Additionally, DNNs can also be used to learn user/item embeddings by leveraging side information such as the item attribute information. For instance, PrivateRec [113] utilizes an attention module and pre-trained models to learn news embeddings, and learns user embeddings based on the history of news clicked by the user. ReFRS [116] utilizes a self-supervised variational autoencoder (VAE) to learn user sequential interaction embeddings. FedCTR [109] proposes to use a neural user model to learn user embedding behaviors across different platforms in a hierarchical manner, thereby obtaining unified user embeddings to improve the performance of CTR prediction. This includes using a multi-head self-attention network to learn representations of user behaviors based on information such as search query behaviors and webpage titles, as well as employing an ad encoder model to learn the embeddings of advertisements. Moreover, there are hybrid methods that combine MF and DNNs to leverage both the linear representational power of MF and the non-linear relational capture ability of DNNs. For example, methods [105, 111] employ NeuMF [133] that ensembles MF and MLP to learn user/item embeddings. Furthermore, DNN-FedRSs can also be applied to the federated cross-domain recommendation tasks. For example, FedCDR [110] utilizes the MLP as a transfer module to facilitate knowledge transfer in the cross-domain recommendation. PriCDR [103] employs the deep auto-encoder and DNN to learn embeddings on source-domain user-item interaction data and target-domain user-item interaction data, respectively. FedCT [100] introduces a modified VAE for both horizontal collaborations between clients and vertical collaborations between platforms for cross-domain recommendations.
- GNN-FedRSs** model user-item interactions as a user-item bipartite graph. Specifically, the data on each client can be viewed as a first-order user-centered ego graph, including only the user and the items they have directly interacted with. In this way, GNNs can be directly

employed to learn user-item embeddings on each client. However, since the graphs on each local device are isolated first-order graphs and data cannot be shared with the server to construct a global graph due to privacy concerns, the full expressive power of GNNs cannot be utilized. To address this issue, SemiDFEGL [28] proposes a semi-decentralized federated recommendation framework, where, on the server side, clients are aggregated into different groups. In each group, fake common items are generated to connect the isolated ego graphs of each client, thereby establishing higher-order local subgraphs. In this way, existing GNN methods can be applied to these local subgraphs to learn higher-order graph structural information through device-to-device collaborative learning. FPPDM [114] employs federated learning to privacy-preserving cross-domain recommendation, i.e., each client is an independent recommendation service platform. Therefore, each client contains a global user-item bipartite graph so that the GNN can be directly utilized to learn the user/item embeddings. FedPerGNN [106] and PerFedRS [108] require an additional trusted third-party server to find and distribute anonymous neighbor information. This is done to expand the local ego graphs into a global graph. In this way, the server can collaborate with many local GNNs for learning. FeSoG [107] utilizes additional social information between users (i.e., user-user connections) to alleviate data sparsity and cold-start issues in the user-item bipartite graph. To this end, attention-based graph neural networks (e.g., GAT [134]) are employed to learn heterogeneous relationship weights, such as user-item and user-user interactions, during neighbor aggregation.

- **RL-FedRSs** consider the recommendation problem as a sequential decision-making problem and employ reinforcement learning techniques to make recommendation decisions based on users' historical interaction data and user/item contexts. For example, FedDSR [112] introduces integrating deep reinforcement learning with curriculum learning to perform daily schedule recommendations. VFUCB [115] employs LinUCB [135] and LinTS [136] to balance exploration and exploitation for online recommendations. To optimize the payload, FCF-BTS [99] adopts a classic reinforcement learning model, i.e., the multi-armed bandit, to intelligently select part of the item embeddings for distribution to all users resulting in reducing the communication costs between server and clients.

4.1.5 Model Upload

After the local training process, each participating client needs to upload locally updated parameters (e.g., $\mathbf{Q}_u$) [110]

or corresponding gradients (e.g., $\frac{\partial \mathcal{L}_u}{\partial \mathbf{Q}_u}$) [88, 93, 112] to the server for the subsequent global model aggregation. Generally, parameters on each device consist of user embedding $\mathbf{p}_u$, item embedding table $\mathbf{Q}_u$, and model parameters Θ_u . It is noteworthy that typically only $\mathbf{Q}_u$ and Θ_u are uploaded to the server, while $\mathbf{p}_u$ remains locally due to privacy considerations. For example, PFedRS [117] proposes to only upload item embedding table $\mathbf{Q}_u$ to the server, and the score function with a MLP parameterized by Θ_u is trained locally. In this way, each client can learn a personalized score function to capture user-specific preferences.

Although most of the current federated recommendation methods could enhance users' privacy due to raw data locality, the transmission of model updates, such as gradients or model parameters, from devices to the central server can still raise additional privacy issues. For example, in each training round, each device only updates the item embeddings corresponding to the items that have interacted with the user. So, the central server can easily recover the user's interaction history by comparing the difference between the newly updated item parameters uploaded by devices and the previous global item embedding table passed to devices [137]. To mitigate this issue, FedRSs typically require some additional techniques to enhance privacy protections. For example, FPPDM [114] proposes to parameterize the locally trained embeddings with a Gaussian distribution to represent the user's preference and only uploads the obtained Gaussian distribution parameters, i.e., mean and covariance, to the global server. HPFL [101] proposes dividing local model parameters into a non-sensitive public component, which can be directly uploaded to the server, and a sensitive private component. The private component is processed by performing a clustering task to obtain cluster centers as drafts representing user preferences, thereby reducing sensitivity. FMFLDP [96] introduces a proxy network to remove metadata and mix it with reports from other users, thereby breaking the linkability of consecutive reports from the same user. Furthermore, there are some widely used privacy protection techniques adopted in the model uploading process, including Homomorphic Encryption (HE), Secret Sharing (SS), Secure Multiparty Computation (MPC), Differential Privacy (DP), Local Differential Privacy (LDP), Fake items (FI), and Mask matrix (MM). We will elaborate these techniques in Section 5.

4.1.6 Global Aggregation

After the model upload process, the server aggregates these parameters/gradients to learn a global model. We refer to this step as the global aggregation. Below, we will introduce

the various global aggregation strategies used by different methods.

- **(Stochastic/Mini-batch) Gradient Descent** [88, 89, 93–95, 97–99, 109]: Upon receiving gradients from users, one of the most straightforward methods is to use gradient descent to update global parameters with $\mathbf{Q}_s \leftarrow \mathbf{Q}_s - \gamma \sum_{u \in \mathcal{U}^+} \frac{\partial \mathcal{L}_u}{\partial \mathbf{Q}_u}$, where $\mathcal{U}^+$ denotes the selected clients in this training round. The approach varies based on the value of $|\mathcal{U}^+|$: when $|\mathcal{U}^+|=1$, it functions as stochastic gradient descent (SGD) [102]; when $|\mathcal{U}^+|=|\mathcal{U}|$, it becomes gradient descent (GD); and when $1 < |\mathcal{U}^+| < |\mathcal{U}|$, it operates as a mini-batch gradient descent method [90].
- **FedAvg** [28, 104, 106–108, 110, 111, 129] is a widely used weighted average aggregation method. Taking the model parameter Θ_u aggregation as an example, FedAvg can be performed as $\Theta_s^{(t+1)} \leftarrow \sum_{u \in \mathcal{U}^+} \frac{|\mathcal{D}_u|}{|\mathcal{D}|} \Theta_u^{(t)}$ for the current time step t , where $|\mathcal{D}_u|$ and $|\mathcal{D}|$ represent the size of the local dataset and the entire dataset, respectively. Additionally, there are other weighted aggregation methods that use different metrics as aggregation weights. For example, HPFL [101] employs the local model's validation accuracy as a dynamic aggregation weight.
- **Average Aggregation** [30, 92, 96, 100, 114, 117] calculates the average value of the parameters uploaded by all participating training clients in each round, which is then used as the global parameters as $\Theta_s^{(t+1)} \leftarrow \sum_{u \in \mathcal{U}^+} \frac{1}{|\mathcal{U}^+|} \Theta_u^{(t)}$.
- **Other Methods:** Federated neural collaborative filtering (FedNCF) [105] introduces a novel aggregation method called MF-SecAvg, which extends the traditional FedAvg [129] by integrating the secure aggregation (SecAvg) protocol [138] with a low computational cost. However, it can only protect the privacy of sampled clients in a single aggregation round. PrivateRec [113] employs FedAdam [139] to update global model parameters based on uploaded gradients. ReFRS [116] proposes a user semantic-based global aggregation approach, aggregating only users with semantic similarities. As mentioned earlier, it first clusters users into different groups and then performs the aggregation in a group-wise manner to achieve quicker model convergence and better personalized recommendation performance. FedDSR [112] proposes a similarity aggregation strategy that aggregates only similar clients. Specifically, it first calculates client-wise similarity based on the gradient parameters uploaded by the clients. Then, it selects clients for global aggregation based on a defined similarity threshold. LightFR [118] proposes a tailored discrete aggregation method for binary matrices, achieved by applying the sign operation on the uploaded binary matrices. FedPerGNN [106]

empirically explores several existing global aggregation methods, including FedAvg [129], FedAtt [140], Per-FedAvg [141], and pFedME [142], and empirically finds that Per-FedAvg and pFedME can achieve better performance. FedFast [91] introduces an efficient federated aggregation strategy called ActcAGG, which involves “delegates” actively participating in training and sharing their progress with “subordinates” who are not involved in the training. This strategy updates non-embedding components similar to FedAvg, and item and user embeddings are updated based on weighted contributions from delegates, with a focus on actively updated components.

4.2 Decentralized Recommender Systems

While FedRS partially alleviates users' concerns about personal data leakage, it presents significant limitations that warrant attention. A primary issue is its heavy reliance on a central server for collecting and aggregating user models, as well as redistributing the combined model. This dependence demands substantial computational and storage resources from the server and exerts considerable strain on network bandwidth, leading to elevated economic and environmental costs. Another critical drawback is the ‘long-tail problem’ arising from all users in FedRS sharing the same model. This results in the aggregated model's inability to effectively capture diverse user preferences.

In response, decentralized recommender systems (DecRSs) have been developed to overcome the limitations of FedRSs. Unlike standard FedRSs, DecRSs optimize on-device models through a combination of local training and inter-device communication within specific user groups. This approach significantly minimizes central server involvement—specifically, the central server primarily assists in assigning neighbors or forming communication groups, while model updates are conducted directly between devices. This is in contrast to FedRSs, where model updates occur between each device and the central server. By avoiding centralized aggregation, DecRSs further reduce privacy risks and enable fully personalized recommendation services for users. In the subsequent sections, we formally define DecRSs and outline their operational pipeline. We also discuss various DecRSs, differentiated by the strategies employed within their pipeline, as summarized in Table 4.

4.2.1 Problem Formulation of DecRSs

Let $\mathcal{U}$ and $\mathcal{I}$ denote a set of users and items, respectively. Each user $u \in \mathcal{U}$ holds a local dataset $\mathcal{D}_u = \{\mathcal{I}_u, \mathbf{y}_u\}$, where $\mathcal{I}_u \subseteq \mathcal{I}$ denotes a set of items interacted with user u , and $\mathbf{y}_u \in \mathbb{R}^{|\mathcal{I}|}$ or $\mathbf{y}_u \in \{0, 1\}^{|\mathcal{I}|}$ represent explicit feedbacks (e.g., ratings) or implicit feedbacks (e.g., clicks and check-ins) of users, respectively. In addition, each

device maintains a local model $f_u(\mathbf{p}_u, \mathbf{Q}_u, \Theta_u)$ that typically consists of parameters including d -dimensional user embedding $\mathbf{p}_u \in \mathbb{R}^d$, item embedding table $\mathbf{Q}_u \in \mathbb{R}^{|\mathcal{I}| \times d}$, and model parameters Θ_u . More importantly, each user can obtain extra knowledge $\mathcal{K}_u$ from the corresponding neighbors $\mathcal{N}_u$. Given these, the goal of DecRSs is to train a local model for each user by minimizing loss functions on the device side $\mathcal{L}_u(\mathcal{D}_u, f_u, \mathcal{K}_u)$.

4.2.2 Typical Pipeline of DecRSs

As shown in Fig. 5, the implementation of DecRSs can be divided into three stages, including (1) Neighbor Identification: for each user, neighbors for collaborative learning are identified with predefined neighbor identification strategies before the training process or at the start of each training round. (2) Local Training: each user trains a local

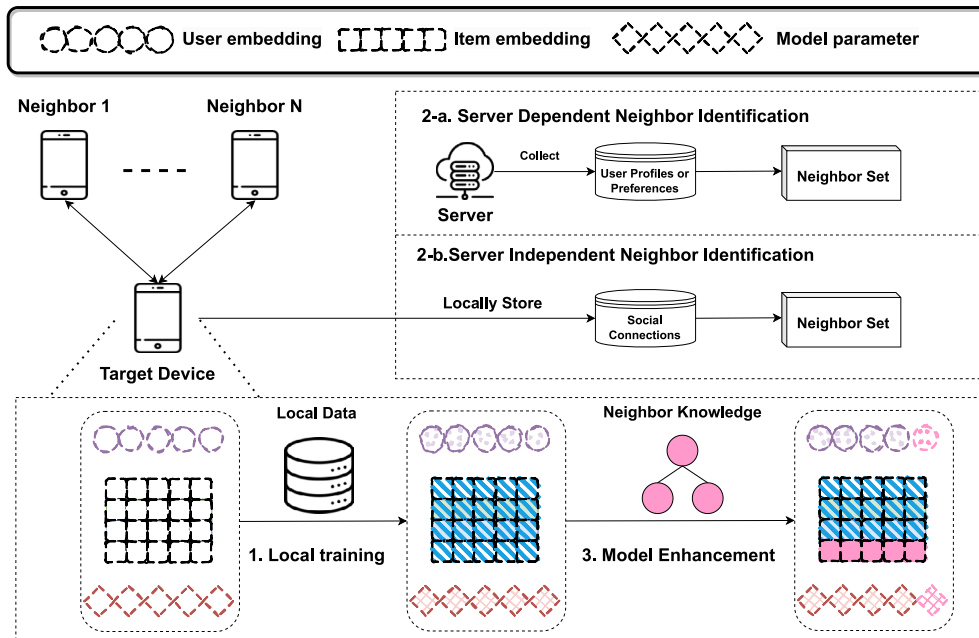


Fig. 5 The pipeline of DecRSs

Table 4 Summary of the strategies used in the different stages of DecRSs

DecRS pipeline	Strategy	References
Local training	MF	[143–151]
	GNN	[28, 152]
	DNN	[27, 29]
Neighbor identification	Social network-based	[29, 143, 150, 151]
	Random assignment	[144, 146, 147, 152]
	General profile-based	[148, 149]
	Geographical information-based	[27, 29, 145, 153]
	User embedding-based	[28, 148, 149]
	Item embedding-based	[148, 149]
	Response-based	[29]
	Category distribution-based	[27, 29]
	Random walk	[144, 145, 151]
	Two-step neighbor identification	[29, 144]
	Performance-triggered sampling	[29]
	Model upload	
Model upload	Data sharing	[144, 148, 149, 153]
	Model Sharing	[27, 28, 143, 146, 147]
	Gradient sharing	[145, 150–152]
	Soft decision sharing	[29]

model with personal data. (3) Collaborative Learning with Neighbors: the locally trained model is further enhanced by exchanging knowledge with neighbors. Strategies used in each stage will be further discussed in the following sections.

4.2.3 Local Training

To facilitate neighbor identification and collaborative learning, each user initially trains a local model using their personal data. Echoing the methodology of FedRSs, the recommendation strategies implemented at the user level predominantly encompass Matrix Factorization (MF), Graph Neural Networks (GNNs), and Decurrent Neural Networks (DNNs). The primary aim of these approaches is to derive insightful user and item embeddings that accurately represent their characteristics and preferences.

4.2.4 Neighbor Identification

Training a recommender system solely on isolated devices faces a severe challenge of data sparsity. Meanwhile, communicating with all other devices poses privacy concerns, high communication costs, and long-tail issues. Hence, DecRSs aim to identify quality neighbors for collaborative learning. Among existing works of DecRSs, neighbor identification approaches can be broadly categorized into two types: (1) **Server-Independent Approaches** and (2) **Server-Dependent Approaches**. This section will comprehensively present various methods under the two categories.

Server-Independent Approaches. In these approaches, neighbor identification is entirely conducted on the user's side. In DecRSs, social information is stored locally, enabling users to interact with their friends. Given that friends often share high preference affinities, considering them as potential high-quality neighbors is a sensible strategy [29, 143, 150, 151]. To expand the scope of communication, users may also connect and exchange information with friends of friends, provided all parties consent [151]. However, relying solely on local social networks for neighbor discovery can be limiting. Quality neighbors with similar preferences may not be part of one's immediate social circle. For instance, users with a shared interest in a specific anime genre are unlikely to be confined to a single social network. Therefore, recent research has shifted towards identifying high-quality neighbors globally.

Server-Dependent Approaches. Under this category, a server is utilized to construct a user adjacency matrix W , where each element $W_{a,b} \in [0, 1]$ indicates the correlation degree between users u_a and u_b . Based on this, the N users most correlated with u_a are chosen as their neighbors, denoted as $\mathcal{N}(u_a)$. The primary challenge lies

in populating this matrix. One basic method is **random assignment**, leading to $\mathcal{N}(u_a)$ being a random subset of the user set $\mathcal{U}$ [144, 146, 147, 152]. It's important to note that this method cannot be implemented locally without access to all device IDs. Beyond random assignment, user correlation is typically quantified based on similarities in profiles and preferences. The specific factors contributing to this correlation will be detailed subsequently:

- **General Profile.** The fundamental components of a user profile usually encompass age, gender, education level, and preference level for various item categories. When users exhibit similar profiles, it implies a shared commonality in their preferences, making them potentially valuable neighbors in recommendation systems. As outlined in [148, 149], the similarity between the profile vectors p_{u_a} and p_{u_b} of two users is quantified as $s_p(u_a, u_b) = \frac{1}{1+d(p_{u_a}, p_{u_b})}$, where $d(\cdot)$ represents a chosen distance function, which could be the Euclidean distance, Manhattan distance, or Cosine distance.
- **Geographical Information.** As a distinctive aspect of user profiles, geographical information plays a pivotal role in categorizing users for recommendation systems. This is due to its significant impact on users' preferences for certain products and services, which can be influenced by factors like climate, transportation, and local culture, all closely tied to geographical location. Such influences are particularly evident in Point-Of-Interest (POI) recommendations, where only users within the same region have access to the same POIs [29]. In light of this, the geographical similarity $s_g(u_a, u_b)$ between two users u_a and u_b is defined as 1 if they are in the same region, and 0 otherwise. Regions can be determined based on administrative boundaries, with adjustable granularity. Furthermore, references [27, 145, 153] propose an alternative quantification of geographical similarity as $s_g(u_a, u_b) = \frac{1}{1+d_{geo}(u_a, u_b)}$, where $d_{geo}(u_a, u_b)$ denotes the physical distance between the users u_a and u_b .
- **User Embedding.** As previously mentioned, each user initially learns user and item embeddings using locally stored data before engaging in collaborative learning with neighbors. These user embeddings, which encapsulate individual preferences, can be utilized to calculate user similarity [148, 149] as $s_u(u_a, u_b) = \frac{1}{1+d(e_{u_a}, e_{u_b})}$, where e_{u_a} and e_{u_b} represent the embeddings of users u_a and u_b respectively, and $d(\cdot)$ is a distance function, such as Euclidean distance, Manhattan distance, or Cosine Distance. Beyond simple pairwise similarity calculations, [28] employs fuzzy c-means clustering to group users based on their embeddings. In this framework,

$s_u(u_a, u_b)$ is assigned a value of 1 if u_a and u_b belong to the same cluster and 0 otherwise.

- **Item Embedding.** Similar to user embeddings, users with similar item embeddings are also considered similar [148, 149]. The item similarity is formally defined as $s_i(u_a, u_b) = \sum_{i \in I_{u_a} \cap I_{u_b}} \frac{e_i^{u_a} \cdot e_i^{u_b}}{\|e_i^{u_a}\| \times \|e_i^{u_b}\|}$, where I_{u_a} and I_{u_b} denote the sets of items interacted with by users u_a and u_b , respectively. It is worth noting that the item similarity is not averaged by the number of overlapping items because the greater number of overlapping items inherently implies higher user similarity.
- **Response on Reference Dataset.** To mitigate privacy concerns associated with uploading personal embeddings, MAC [29] proposes using soft decisions derived from applying the personal model Θ to a public reference dataset $\mathcal{D}^{ref}$. This approach provides a less sensitive yet effective representation of user preferences. User similarity in this context is quantified using the Kullback–Leibler (KL) divergence as $s_r(u_a, u_b) = \sum_{\forall \mathcal{X} \in \mathcal{D}^{ref}} KL(\Theta_a(\mathcal{X}) || \Theta_b(\mathcal{X}))$.
- **Category Distribution.** In recommendation systems, the prevalent issue of data sparsity often makes it difficult to identify a sufficient number of high-quality neighbors based solely on overlapping items. An alternative approach is to consider the categories of items interacted with by users. These category distributions, being denser and less sensitive than individual items, offer a secure way to reflect similarities in user preferences. In this context, [27, 29] employ the Kullback–Leibler (KL) divergence to measure the category similarity as $s_c(u_a, u_b) = KL(CP(u_a) || CP(u_b))$, where $CP(u_a)$ and $CP(u_b)$ represent the distributions across all item categories for users u_a and u_b , respectively.

After collecting the corresponding information, the server can build the user adjacency matrix W by calculating and normalizing user similarities. In addition, a **random walk approach** can be further utilized to refine the matrix [144, 145, 151]. This method capitalizes on the principle of transitive correlations among users: if u_a and u_b exhibit a high degree of correlation, and similarly, u_b and u_c are closely correlated, it is reasonable to infer a significant correlation between u_a and u_c . The random walk algorithm navigates through these interconnected user relationships, enabling the system to identify and reinforce these indirect yet meaningful correlations. Consequently, the adjacency matrix becomes more robust, accurately reflecting the nuanced network of user correlations, both direct and indirect.

While proven effective in identifying neighbors, existing methods also present notable drawbacks, broadly falling into two categories. Firstly, the quality of neighbors identified by some methods varies, leading to less effective

collaborative learning. A prime example is the random-based method, which often pairs users with neighbors having little relevance to their interests. Moreover, even if users share similar profiles, geographical locations, or category distributions, their product preferences may not fully align. Consequently, not all neighbors identified by these methods are genuinely meaningful. Secondly, some methods that can accurately define high-quality neighbors necessitate users to share sensitive information (such as personal embeddings or even soft decisions) with the server, posing significant privacy concerns. In summary, there remains an unmet need for a method capable of identifying high-quality neighbors without compromising user privacy.

To bridge this gap, a **two-step neighbor identification** method is proposed [29, 144]. Initially, a server utilizes non-sensitive information (e.g., de-identified geographical information and category distributions) to identify a preliminary set of neighbors. The user device then refines this set using more sensitive yet representative data. This approach enables users to obtain high-quality neighbors for collaborative learning while minimizing privacy risks. In addition, the fluctuations in the local loss function are valuable indicators of the current neighbors' informativeness. Thus, [29] introduces a **performance-triggered sampling** strategy. A small subset of neighbors is initially sampled from the pre-selected pool. During each collaborative learning (CL) epoch o , if the change in $\Delta L_{loc}^o(u_a)$ is less than a predefined threshold $\tau = 1\%$, the neighbors are reselected from the pre-selected set. The change in local loss, $\Delta L_{loc}(u_i)$, is defined as $\Delta L_{loc}^o(u_i) = \frac{|L_{loc}^o(u_i) - L_{loc}^{o-1}(u_i)|}{L_{loc}^{o-1}(u_i)} \times 100\%$, where the τ is adjustable and controls the sensitivity to performance changes.

4.2.5 Collaborative Learning with Neighbor

Given the knowledge from high-quality neighbors $\mathcal{N}_a$ and the corresponding similarities, the next step is to enhance the personalized model Θ_a with such information. According to different knowledge shared between users, there are currently four types of collaborative learning approaches: data sharing approach, model sharing approach, gradient sharing approach, and soft decision sharing approach. To avoid repetitive narration, the common symbols used in this section are listed here: γ is the learning rate, μ controls the strength of the neighbor knowledge, $\mathcal{W}_{ab}$ represents the similarity of u_a and u_b .

Data Sharing Approach. The straightforward approach is to share data (i.e., ratings or interacted items) with neighbors for collaborative learning [144, 148, 149, 153]. Upon receiving the data from the neighbors, the personal model can be further updated with SGD, formulated as $\Theta_a \leftarrow \Theta_a - \gamma \mu \frac{1}{|\mathcal{N}_a|} \sum_{u_b \in \mathcal{N}_a} \mathcal{W}_{ab} \frac{\partial \mathcal{L}(\Theta_a, \mathcal{D}_b)}{\partial \Theta_a}$, where $\mathcal{L}(\cdot)$ is either

$\mathcal{L}_{MSE}$ for ratings or $\mathcal{L}_{CE}$ for interacted items, and $\mathcal{D}_b$ refers to personal data of u_b .

Model Sharing Approach. Even within a small scope, directly sharing private data carries significant risks of privacy breaches. On this basis, recent studies of DecRSs allow users to share models, keeping private data on the user side consistently [27, 28, 143, 146, 147]. Given the neighbor models, the enhanced model is defined as $\Theta_a \leftarrow \Theta_a + \mu \frac{1}{|\mathcal{N}_a|} \sum_{u_b \in \mathcal{N}_a} \mathcal{W}_{ab} \Theta_b$.

Gradient Sharing Approach. To further mitigate the risk of privacy leakage, gradients are utilized as the bridge for collaborative learning instead of the model [145, 150–152]. Then, the personal model enhanced by such information is formulated as $\Theta_a \leftarrow \Theta_a - \gamma \mu \frac{1}{|\mathcal{N}_a|} \sum_{u_b \in \mathcal{N}_a} \mathcal{W}_{ab} \frac{\partial \mathcal{L}_b}{\partial \Theta_b}$. However, some methods [154, 155] successfully reconstructed the model with all gradients, proving that sharing models and sharing gradients possess nearly the same level of privacy security and collaborative effectiveness.

Soft Decision Sharing Approach. Unfortunately, frequently transporting models or gradients breaks the purpose of communication efficiency. In addition, both approaches have a strong assumption that all users have homogeneous models. This assumption significantly harms the practicality of DecRSs as each local model should have a customized structure to meet the specific device. In light of this, MAC [29] proposes to let u_a get close to her neighbors by minimizing their disagreement of soft decisions on the public reference dataset $\mathcal{D}^{ref}$, which is quantified as $L_{SD} = \frac{1}{|\mathcal{N}_a|} \sum_{u_b \in \mathcal{N}_a} \mathcal{W}_{ab} \left(\sum_{\mathcal{X} \in \mathcal{D}^{ref}} \|\Theta_a(\mathcal{X}) - \Theta_b(\mathcal{X})\|_2^2 \right)$. Then, the enhanced model is defined as $\Theta_a \leftarrow \Theta_a - \gamma \mu \frac{\partial L_{SD}}{\partial \Theta_a}$.

4.3 On-Device Recommender Finetuning

Although device-cloud collaborative learning (FedRSs) and device-device collaborative learning (DecRSs) have demonstrated their ability to provide accurate yet secure recommendations immediately, they both require users to have sufficient computation resources to participate in the training process, impairing their practicality. Moreover, the extensive training time might also annoy inactive users. To

solve this problem, researchers proposed another on-device learning approach known as on-device recommender fine-tuning. Specifically, a server is initially involved in training a global model and distributing the global model to all user devices. Then, this model is adapted to each user by fine-tuning with local data. Since the deployment approach is well-discussed in existing studies, which are discussed in Section 3, researchers in this domain focus on making the deployed model adapt to user data quickly and effectively. In summary, three types of fine-tuning methods will be discussed in this section and demonstrated in Fig. 6. To reduce redundancy, the common notations in this section are listed here: γ is the learning rate, $\mathcal{L}(\cdot)$ is either $\mathcal{L}_{MSE}$ for ratings or $\mathcal{L}_{CE}$ for interacted items, and $\mathcal{D}_a$ is the local data of u_a .

Whole Model Finetuning. The most straightforward way is to update the whole network with the local data [30, 32, 156]. Formally, given the deployed model Θ_s pre-trained on the server, the personalized model on the device side for u_a is defined as $\Theta_a \leftarrow \Theta_s - \gamma \frac{\partial \mathcal{L}(\Theta_s, \mathcal{D}_a)}{\partial \Theta_s}$. However, given the extensive scale parameters involved in the cloud model, adapting the entire network would be an impractical approach.

Partial Model Finetuning. Alternatively, method [157] proposes to finetune partial parameters. That is, after u_a receives the pre-trained model Θ_s , each item embedding can be decomposed into two parts: $e_i = e_i^s + e_i^a$, where e_i^s represents stable parameters, while e_i^a denotes the variable part and its size can be adjusted. Then, e_i^a is updated with the local data as $e_i^a \leftarrow e_i^a - \gamma \frac{\partial \mathcal{L}(\Theta_s, \mathcal{D}_a)}{\partial e_i^a}$. Unfortunately, only finetuning partial parameters is performance-limited as the awkward combination of stable and variable vectors disrupted the original features.

Patch Learning-based Finetuning. Hence, patch learning is introduced to address the shortcomings of partial model finetuning, which has been proven to achieve comparable performance with whole model finetuning [31]. Concretely, after deploying the pre-trained model Θ_s on the device of u_a , a personalized patch model Ψ_a is inserted and the adjusted model is defined as $\Theta_a(\cdot) \leftarrow \Theta_s(\cdot) + \mu \Psi_a(\Theta_s(\cdot))$, where μ controls the level of the adaptation effect. Then, only the patch model is uploaded with the local data by

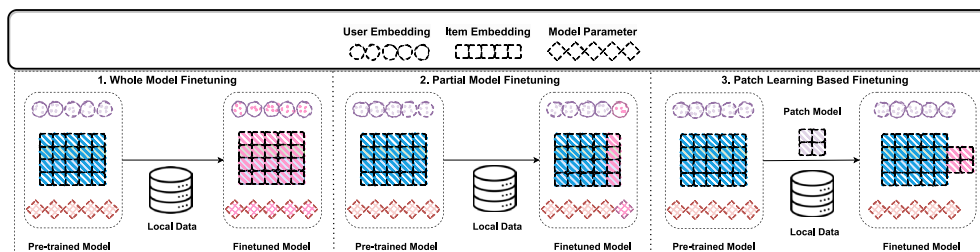


Fig. 6 Three types of on-device finetuning methods

$\Psi_a \leftarrow \Psi_a - \gamma \frac{\partial \mathcal{L}(\Theta_a, D_a)}{\partial \Psi_a}$. Note that, the patch model could have different neural architectures if its inputs and outputs are aligned with Θ_s .

5 Security for DeviceRSs

With the notable accomplishments that DeviceRSs have achieved in both academia and industry in recent years, numerous researchers have embarked on exploring DeviceRSs the perspective of security standpoint. Generally, security studies related to DeviceRSs encompass two crucial aspects: safeguarding privacy and ensuring the robustness of systems. In the subsequent subsections, we present a thorough literature review to introduce security research undertaken in these two dimensions.

5.1 Privacy Risks and Countermeasures

Within the context of DeviceRSs, there exist two primary participant categories: users and service providers. These distinct participant types harbor entirely different privacy requirements. Figure 7 delineates the privacy considerations in DeviceRSs as perceived by these diverse participants. For users, paramount privacy concerns revolve around the protection of their behavioral data, sensitive information, and data management rights. In contrast, service providers place their privacy emphasis predominantly on model information and model intellectual property rights. In the ensuing discussion, we will delve into privacy considerations within DeviceRSs from both the users' and service providers' perspectives.

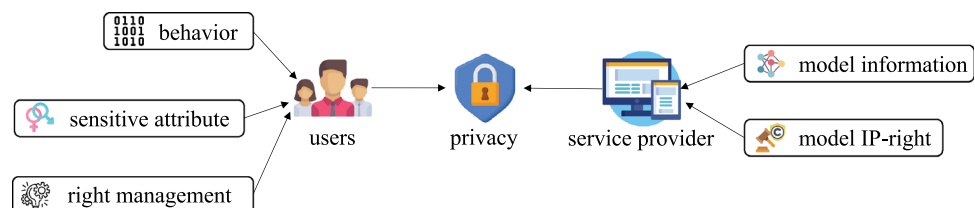
5.1.1 Privacy Issues and Solutions from User Perspectives

Compared to CloudRSs, DeviceRSs shift the training process to the devices' side considering user privacy. For example, in a typical DeviceRS (such as a federated recommender system), users' raw data are consistently stored on their respective local devices and remain inaccessible to other participants, and the model is trained based on local data while aggregate parameters with other participants to achieve collaborative learning. Despite the privacy-centric design of DeviceRSs, numerous studies have delved into and

verified their actual privacy protection capabilities. These investigations have revealed that, in practice, DeviceRSs encounter several challenges related to user privacy.

- User Behavior Data Leakage.** User behavior data encompass both explicit behaviors, such as user review ratings, and implicit information, including clicking, browsing, and purchasing. When recommending based on explicit feedback, where uploaded item embedding gradients pertain solely to users' rated items, a central server with inquisitive intentions can easily identify rated items by analyzing non-zero gradients [158]. Furthermore, FedMF [89] indicates that by scrutinizing gradients sent by clients over two consecutive rounds, the central server can even infer the rating scores of items, leading to compromised rating information within the current mechanism. For recommendation systems using implicit feedback, all non-interacted items are treated as negative samples, and the uploaded item gradients encompass both user-interacted and non-interacted items. Consequently, these methods appear to offer enhanced privacy protection. However, Yuan et al. [33] reveal that a curious but honest central server can accurately deduce users' interacted item sets by calculating the similarity between mimic item embeddings trained on a guessing dataset and the original uploaded item embeddings.
- User Sensitive Attribute Leakage.** Except for user behavior information, customers in recommender systems have other personal attributes that need to be protected, including but not limited to gender, age, occupation, political orientation, health status, etc. [159]. In the context of DeviceRSs, where the interaction data is maintained by clients and the central server lacks access to user recommendation lists, conventional attribute inference attack methods designed for centralized recommender systems are inapplicable [159, 160]. The pioneering work of [36] delves into the potential threats of attribute leakage in DeviceRSs. Their research highlights that in the presence of a small proportion of data leakages, a curious-but-honest central server can train an attacker with a minimal number of Multilayer Perceptron (MLP) layers to achieve high inference accuracy.
- Users' Data Ownership.** Most recent privacy regulations [161] have claimed that users have the right to control and withdraw their data contributions at any time,

Fig. 7 Different perspectives of privacy in DeviceRSs



named “The Right to Be Forgotten”. Unfortunately, the work [35] suggests that there is a lack of consideration for effectively removing the contributions of the user. In light of this, FRU [35] employs a federated unlearning method to efficiently remove specific clients’ influence via historical update calibration.

As shown in the above description, although DeviceRSs offer certain privacy protection for users, they still have many privacy issues. To further improve the privacy-preserving ability, many works incorporate various privacy protection mechanisms in DeviceRSs. These methods can generally be classified into two types: obfuscation-based approaches and encryption-based approaches.

Obfuscation-based Protection. The fundamental concept behind obfuscation-based methods is to perturb users’ data or model parameter updates, thereby rendering it challenging for adversaries to easily infer sensitive information. Broadly speaking, obfuscation-based protection can be categorized into two types: data obfuscation protection and model obfuscation protection.

- **Data Obfuscation.** As mentioned before, Wu et al. [158] have identified that user-interacted items can be detected based on non-zero gradients in recommendations from explicit feedback. Therefore, Lin et al. [97] have devised a hybrid filling strategy to generate synthetic ratings for select items. However, employing such methods excessively can significantly decrease model performance. FedRS++ [98] addresses this issue by utilizing a denoising client group to summarize noisy gradients, assisting the central server in mitigating side effects from fake items. FeSoG [107] employs pseudo-labeling techniques with item sampling for privacy protection. Additionally, some works [111, 128] use fake marks to enhance privacy, generating fake gradients for unrated items and fabricating fake ratings.
- **Model Obfuscation.** Model obfuscation refers to the method that adds noise to the model gradients or model parameters. Common methods for model obfuscation are the use of Differential Privacy (DP) and Local Differential Privacy (LDP) [162] when sharing model parameters. For example, F2MF [104] proposes to use DP techniques to protect users’ group information, thereby achieving the fairness objective in the federated recommendation. PrivateRec [113] introduces a differentially private attention module to provide a differential privacy (DP) guarantee for uploaded decomposed user embeddings. DP-PrivRec [30] utilizes Gaussian mechanism [163] to perturb the uploaded gradients and introduces an adaptive gradient clipping mechanism for controlling the contribution of each client. ReFRS [116] proposes only to upload the parameters of the encoder to the server, and additional

DP and homomorphic encryption (HE) techniques are integrated with the parameters for privacy concerns. FedCTR [109], in the context of multi-platform federated learning, employs LDP and DP techniques to encrypt the local user embeddings on each platform and the aggregated user embeddings on the server side, respectively. Both [158] and [92] implement Laplace Noise-based LDP in FedRSs by clipping gradients and introducing Laplace noise. Liu et al. [107] explore the addition of a dynamic scale of noise to different gradients based on their magnitudes. Zhang et al. [36] highlight that different model components exhibit varying vulnerabilities to inference attacks. In response, they propose an adaptive mechanism to establish a dynamic privacy budget for each model component. Mask matrix is also widely used in DeviceRSs. It obfuscates gradients or parameters by multiplying them with a random matrix. For example, VFUCB [115] introduces an orthogonal matrix-based mask mechanism (O3M), which masks the shared user feature data by multiplying with an orthogonal matrix under the cross-platform federated recommendation scenarios. Analogously, FedNCF [105] generates random matrices based on random seeds shared between clients, and these matrices are used as masks by multiplying them with the uploaded parameters. FedeRank [94] allows users to freely control the proportion of the gradient they upload, with the remaining portion being masked as zero.

To sum up, whether employing data obfuscation or model obfuscation, additional noise is introduced during the training process. Consequently, all of these methods must strike a balance between system utility and privacy protection.

Encryption-based Protection. Another way to protect user privacy is to use advanced encryption algorithms to encrypt parameters before uploading them to the central server. In general, there are three commonly used encryption schemes: homomorphic encryption [164], secret sharing [165], and secure multiparty computation (MPC) [90].

- **Homomorphic Encryption.** Homomorphic encryption is a form of encryption that enables computational operations on encrypted data without the need for decryption [164]. In DeviceRSs, this kind of encryption technique is used to encrypt public parameter gradients [89] or private parameters [108, 128, 158, 166, 167]. In general, homomorphic encryption can protect users’ privacy while maintaining system utility. However, it has some limitations. The first limitation pertains to ensuring the safety of secret keys. The second drawback is that encryption algorithms typically increase computational burden; therefore, these encryption methods may not be applicable to certain edge devices.

- **Secret Sharing.** Secret sharing [165] breaks a “secret” into multiple pieces and disperses these pieces to multiple participants so that the secret can only be reconstructed when all pieces are gathered. Li et al. [168] employ secret sharing to compute the similarity between every pair of rated items, thus safeguarding the privacy of users’ rating values. Ying et al. [169] apply secret sharing to protect both item ratings and user interaction behavior. FR-FMSS [128] and FMSS [111] combine both fake items and secret sharing to provide more comprehensive privacy protection. Compared to obfuscation-based methods and homomorphic encryption, secret sharing can provide privacy protection with lower model performance compromises and lower computation power; however, the distribution of secret pieces will result in huge communication costs.
- **Secure Multiparty Computation (MPC)** [90]: MPC allows multiple clients/platforms to jointly compute a function over their inputs while keeping those inputs private. For example, S³Rec [95] employs MPC and private information retrieval [170] techniques to ensure the secure matrix multiplication protocol for the cross-platform federated social recommendation.

5.1.2 Privacy Issues and Solutions from Service Provider Perspectives

Most current privacy-related works in DeviceRSs mainly investigate how to fulfill users’ privacy needs as we discussed in Section 5.1.1. However, as illustrated in Fig. 7, service providers also have privacy concerns when using DeviceRSs. Specifically, in DeviceRSs, recommender models are exposed to all participants as they are deployed on customers’ local devices. Since recommender models are the core intellectual property and the service providers develop them with substantial expenses, especially in commercial scenarios, the guarantee of model information protection is crucial to convince service providers to utilize DeviceRSs. Besides, how to protect the model IP-right after deploying DeviceRSs is also important for service

providers. PTF-FedRec [171] is a model parameter-transmission-free FedRS framework, allowing the clients and the central server to collaboratively learn heterogeneous models. Based on their framework, the service provider can assign some naive models and algorithms on the client side while deploying the powerful model on the server side. Overall, the protection of model information and model IP-right are still under-explored in DeviceRSs.

5.2 Poisoning Attacks and Countermeasures

The robustness problem mainly exists in the training process, where adversaries can influence the model using pollution. In the traditional CloudRSs, the threats for robustness mainly exist in the data integrating stage, where adversaries can manipulate the recommender system to achieve specific malicious goals (e.g., promoting or demoting specific items with financial incentives [172], degrading the performance of the system [173], etc.) by uploading fake item interactions into the training dataset [174–178]. This kind of attack is called a data poisoning attack.

In DeviceRSs, as the users can directly upload model parameters to update the recommender model, the threats for robustness are more complicated. Specifically, the adversary can not only craft fake interactions to disturb the training process, but also directly upload poisoned model updates to manipulate the system, namely model poisoning attacks. In the following, we introduce the research of both data poisoning attacks and model poisoning attacks and their corresponding solutions. It is worth mentioning that the current attack and defense works of DeviceRSs mainly focus on federated recommender systems. The robustness of other kinds of DeviceRSs, such as semi-decentralized DeviceRSs are still under-explored. Figure 8 compares different types of attacks in CloudRSs and DeviceRSs.

5.2.1 Data Poisoning Attacks and Defense

Research on data poisoning attacks in the context of DeviceRSs is considerably less than that on model poisoning

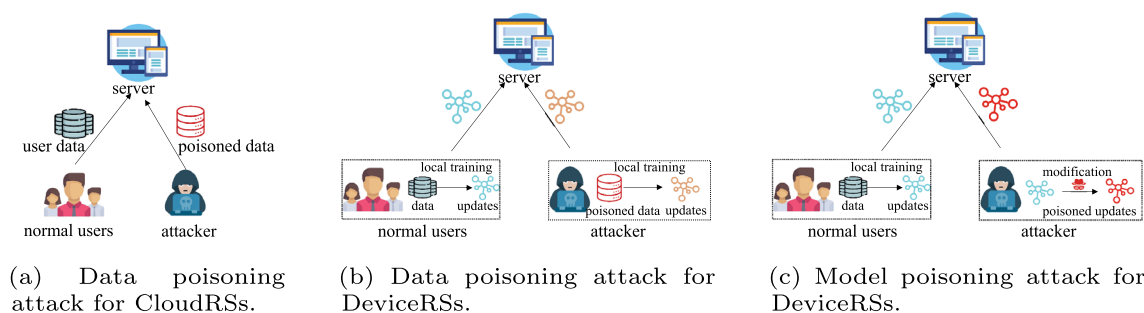


Fig. 8 Overview of different attacks for CloudRSs and DeviceRSs

attacks. This phenomenon can be attributed to several reasons. Firstly, traditional data poisoning attacks, such as Bandwagon attacks [179, 180], can only be executed under the assumption that a portion of user private data is accessible. However, in DeviceRSs, launching these data poisoning attacks is more costly than in centralized recommender systems, given that users' data are individually managed by devices. To acquire these data, adversaries would need to compromise a large number of devices. Besides, some studies suggest that these data poisoning attacks are more detectable in DeviceRSs by analyzing the gradients uploaded by clients. For example, Jiang et al. [181] design a set of gradient features, encompassing gradient deviation from average agreement, weighted gradient degree, gradient similarity with neighbors, and Filler mean difference. Then, based on these features, a data poisoning attacker (e.g., HySAD [182]) can be detected using clustering methods. FedAttack [38] stands out as the sole pure data poisoning attack specifically designed for a typical kind of DeviceRSs (FedRSs) so far. The goal of FedAttack is to disrupt a federated recommender system with limited Byzantine clients. It leverages the observation that globally hardest negative samples are usually detrimental to the system as they have a high chance of being false positive items and guiding the model to converge to local minima [183].

5.2.2 Model Poisoning Attacks and Defense

Model poisoning attacks are a specific category of attacks designed for DeviceRSs. Exploiting the characteristics of DeviceRSs, which allows all participants to access the recommendation model, adversaries in model poisoning attacks craft poisoned model updates to directly modify the model.

PipAttack [37] stands as the initial exploration into model poisoning attacks in FedRSs. It achieves item promotion by aligning the embeddings of target items with popular items, requiring prior knowledge of these popular items. A drawback of PipAttack is its reliance on controlling a substantial number of malicious clients to ensure effectiveness. To reduce the number of adversarial clients, FedRSAttack [184] permits attackers to acquire a portion of benign clients' interaction data, using this data to mimic benign user embeddings—an assumption with potential limitations. Both PipAttack and FedRSAttack rely on specific prior knowledge beyond the generic learning protocol, limiting their applicability in various scenarios. In response to these limitations, Rong et al. [185] propose two model poisoning attacks, A-ra and A-hum, which do not depend on any prior knowledge. A-ra employs random vectors sampled from a Gaussian distribution to simulate the feature vectors of normal clients, while A-hum further optimizes these random vectors by treating the target items as negative samples to create “hard users”. PSMU [39] suggests that incorporating alternative

items can enhance the competitiveness of target items. Yuan et al. [186] take the pioneering step of combining data poisoning attacks and model poisoning attacks in visually-aware FedRSs. They promote target items by contaminating both their visual signals (e.g., item posters) and item embeddings, highlighting the potential threats of incorporating third-party images in FedRSs. In contrast to the aforementioned model poisoning attacks, whose adversarial objective is to manipulate the rank of specific items, Yu et al. [173] aim to diminish overall model performance by uploading gradients to guide item embeddings to converge into several dense clusters. Yin et al. [187] propose a targeted poisoning attack, PoisonFRS, which injects fake users into federated recommender systems without requiring any knowledge of genuine user data or server aggregation rules, and carefully crafts model updates based solely on item embeddings to promote attacker-chosen items to a large portion of genuine users.

All of the above model poisoning attacks, especially those with more realistic setting [39, 185, 186], consistently underscore the vulnerability of DeviceRSs to model poisoning attacks, attributed to the exposure of the training process. To address this security loophole, numerous defense methods have been researched recently.

Some works [37, 39, 173] have explored defense methods proposed in federated learning. For instance, Zhang et al. [37] applies Bulyan [188] and Trimmed Mean [189] to defend against their proposed PipAttack. The experimental results showcase that these two defenses cannot achieve robust performance. PSMU [39] further employs an item-level Krum [190], Median [189], and gradient clipping [188] to safeguard recommender systems. Nevertheless, their research suggests that Krum, Median, and Trimmed Mean significantly compromise model performance. Yu et al. [173] delve into the reasons behind the ineffectiveness of these federated learning defense mechanisms in DeviceRSs. The primary challenge lies in the non-IID (non-identically distributed) nature of data for each client, leading to significant gradient deviations for both benign and malicious clients. Consequently, distinguishing poisoning gradients in DeviceRSs becomes a formidable task. The work [191] is one of the pioneers studying the defense method to improve the robustness of DeviceRSs to resist model poisoning attacks. It proposes A-FRS and A-RFRS to improve Byzantine resilience, however, these defenses are only shown to protect models from gradient ascent attack [190]. HiCS [39] introduces a sparsification update with gradient clipping defense methods which can defend against most model poisoning attacks. Liang et al. [192] propose a contribution-aware robust aggregation scheme, ConDA, which estimates client contributions in a decentralized manner using ensemble-based Shapley values and integrates them via majority voting into a Bayesian knowledge distillation framework to defend FedRSs against untargeted attacks. However, this

method may slow down the convergence of models. Yu et al. [173] design a defense named UNION to improve the resilience of FedRSs for untargeted model poisoning attacks.

6 Evaluation Metrics

The evaluation of on-device recommender systems (DeviceRSs) encompasses multiple dimensions beyond standard accuracy. Due to the resource-constrained nature of edge devices and the increasing concern over user privacy, performance metrics must comprehensively reflect accuracy, efficiency, communication cost, and robustness. In this section, we organize the evaluation metrics into four categories: general recommendation accuracy, on-device inference efficiency, training and update cost, and robustness against security and privacy threats.

6.1 General Recommendation Accuracy

Accuracy remains the most fundamental measure for recommender systems, with tasks generally falling into three categories: ranking (e.g., top- K recommendation), regression (e.g., rating prediction), and classification (e.g., click-through rate prediction).

- **Hit Ratio@K (HR@K):** Measures whether the ground-truth item appears in the top- K ranked list. It reflects the system's recall capability at a fixed cutoff.

$$HR@K = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}(r_u \in R_u^K), \quad (1)$$

where U is the set of users, r_u is the ground-truth item for user u , and R_u^K is the top- K recommendations.

- **Recall@K:** Captures the proportion of relevant items successfully retrieved in the top- K list. It is especially relevant when users may have multiple relevant items.

$$Recall@K = \frac{1}{|U|} \sum_{u \in U} \frac{|R_u^K \cap T_u|}{|T_u|}, \quad (2)$$

where T_u is the ground-truth item set for user u .

- **Normalized Discounted Cumulative Gain (NDCG@K):** Takes into account the position of relevant items in the top- K list, assigning higher weight to items ranked higher.

$$NDCG@K = \frac{1}{|U|} \sum_{u \in U} \frac{DCG_u@K}{IDCG_u@K}, \quad DCG_u@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}, \quad (3)$$

where rel_i denotes the binary relevance of the item at position i , and $IDCG_u@K$ is the ideal DCG score.

- **Mean Squared Error (MSE):** Commonly used in rating prediction tasks, MSE measures the average squared difference between predicted scores and ground-truth ratings.

$$MSE = \frac{1}{|D|} \sum_{(u,i) \in D} (\hat{r}_{ui} - r_{ui})^2, \quad (4)$$

where D is the set of user-item pairs, and $\hat{r}_{ui}$ and r_{ui} denote the predicted and actual ratings respectively.

- **AUC (Area Under the ROC Curve):** Used primarily in binary classification tasks such as click-through rate (CTR) prediction. AUC reflects the probability that a randomly selected positive instance is ranked above a negative one.

6.2 On-Device Deployment and Inference Efficiency

DeviceRSs are designed to operate under strict resource constraints. Therefore, evaluation of model size and inference time becomes critical.

- **Model Size:** Measures the storage footprint of the model. Smaller models are better suited for mobile and embedded environments with limited storage and memory.
- **Inference Time:** The average time required to produce recommendations for a single user or session on-device.

6.3 Training and Update Efficiency

On-device recommender systems often rely on federated or decentralized training frameworks to avoid transmitting raw user data. In this context, communication cost and convergence time are widely used evaluation metrics.

- **Communication Cost:** Refers to the volume of data transferred between client devices and the central server during each training round. It includes model updates, gradients, or compressed representations.
- **Convergence Time:** Indicates the number of training rounds or total elapsed time required to reach a performance plateau.

6.4 Security and Privacy Robustness

In federated and on-device settings, user data is kept local, but model updates may still leak private information. Robustness against inference attacks and adversarial manipulations is thus a critical evaluation dimension.

- **F1 Score (Attribute Inference Attack Resistance):** Measures the ability of an attacker to infer sensitive user attributes (e.g., gender, age) from exposed model updates

or embeddings. A lower F1 score from the attacker's classifier implies higher privacy protection by the system.

- **Exposure Rate (ER@K):** Designed to evaluate a system's vulnerability to poisoning or promotion attacks. In such attacks, an adversary aims to manipulate the model so that a specific *target item* appears more frequently in user recommendation lists. ER@K quantifies how often the target item is exposed among the top- K ranked items:

$$ER@K = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}(t \in R_u^K), \quad (5)$$

where t is the adversarial target item, and R_u^K is the top- K recommendation list for user u . A high ER@K score suggests that the model is more susceptible to targeted exposure, indicating lower robustness.

7 Challenges and Further Directions

Although DeviceRSs have shown promising performance, we will discuss the challenges they still face and potential future research directions.

7.1 Heterogeneity in DeviceRSs

Most DeviceRS methods assume that each device/user is homogeneous, but this assumption is difficult to satisfy in real life due to the inherent heterogeneity among devices/users, which mainly includes data heterogeneity, system heterogeneity, and privacy heterogeneity. Data heterogeneity refers to the distribution of data on each client side, which is heterogeneous, usually non-independent and identically distributed [193]. System heterogeneity refers to each device's performance, e.g., storage, computation, and communication capabilities are different. Overlooking the heterogeneity will inevitably degrade the performance of the model [194]. Privacy heterogeneity refers to users could have a different privacy budget, with some users not minding sharing their private data with platforms in exchanges for better recommendation service [94, 195].

7.2 Fairness in DeviceRSs

On top of the heterogeneity challenge mentioned above, another future challenge that needs to be focused on is the fairness of DeviceRSs [196]. For example, in federated recommendation methods, when utilizing FedAvg, clients with larger datasets are typically assigned larger aggregation weights. This leads to a bias where recommendations are skewed in favor of those clients with more data. As a result, the system may not accurately represent or serve the interests of clients with smaller datasets, potentially impacting

the overall effectiveness and fairness of the recommendation system.

7.3 Evolving User Dynamics in DeviceRSs

Most current DeviceRSs assume that the number of users remains constant, which is often not the case in real-life scenarios. To address this, future research should focus on two main areas: (1) Cold start for DeviceRSs: the cold-start problem in DeviceRSs presents a more complex challenge compared to traditional CloudRSs. While CloudRSs focus on generating recommendations for new users or items with minimal historical data, DeviceRSs face a new additional task of deploying models to newly added devices. This dual challenge involves not only providing accurate recommendations for cold-start users or items but also ensuring rapid model deployment, coupled with the requirement to cater to varying device capabilities. (2) Unlearning for DeviceRSs: When users exit the system, it is essential to handle their data appropriately. This is where "federated unlearning [35]" comes into play, a process of selectively forgetting or disregarding the data of users who are no longer active. This approach not only maintains the accuracy and relevance of the system's recommendations but also addresses privacy concerns.

7.4 Model Copyright Protection in DeviceRSs

Unlike CloudRSs, DeviceRSs expose recommender models to all users. In practical scenarios, especially in commercial applications, recommender models represent core intellectual properties (IP) that entail significant efforts and financial investments from various experts. Therefore, it is crucial to explore methods for protecting Model IP in DeviceRSs settings. This protection can be approached from two aspects: (1) preventing the model copy behavior and (2) detecting illegal model copy behavior. [171] delves into preventing model copy behavior by achieving heterogeneity in clients' and the server's models. Additionally, encryption methods can be considered to thwart illegal model extraction. To detect illegal model copy behavior, one potential approach involves embedding watermarks in recommender models.

7.5 Foundation Models in DeviceRSs

Inspired by the remarkable progress of foundational models across various research domains, such as natural language processing [197] and computer vision [198], there is growing interest in developing foundational models specifically for recommender systems [199]. Most existing efforts [200–203] are based on cloud-centric architectures, which inherently face challenges such as response latency. However, due to their large-scale nature, transferring these

foundational models to on-device recommender systems (DeviceRS) is nearly infeasible with current techniques. Future research could explore lightweighting these models and leveraging cloud-edge-device collaboration [204, 205] to efficiently deploy them, addressing scalability challenges while maintaining performance.

7.6 Quantitative Evaluation and Benchmarking in DeviceRSs

Current research on DeviceRSs is conducted using various datasets and evaluated through diverse evaluation metrics (e.g., recall, communication costs, memory consumption), experimental settings (e.g., leave-one-out validation, split-by-ratio for training, validation, and testing), and tasks (e.g., top-k recommendation, sequential recommendation, CTR prediction). However, the absence of a standardized benchmarking framework presents significant challenges in comparing these systems and deriving consistent conclusions about their performance. Although recent efforts such as BARS [206] and Tenrec [207] have made important progress toward open benchmarking for general recommender systems, there remains an urgent need for a dedicated and comprehensive evaluation framework that unifies tasks and experimental settings specifically for DeviceRSs [208], enabling more consistent and fair comparisons.

8 Conclusion

The migration of cloud-based recommender systems (CloudRSs) to device-based recommender systems (DeviceRSs) has become a research trend in both academia and industry. In this survey, we comprehensively reviewed research on DeviceRSs from three aspects: model deployment and inference, training and updating, as well as security and privacy. For each aspect, we systematically categorized the existing methods, summarized the main architectures, and outlined the strategies adopted by different methods. Additionally, we discussed the current challenges in this research area and introduced potential future research directions. We hope that this review will provide readers with a thorough understanding of this field.

Acknowledgements Not applicable.

Author Contributions Hongzhi Yin proposed the original idea for the survey, led the design of the taxonomy, and was responsible for the overall revision of the manuscript. Liang Qu contributed to writing the *Training and Updating* sections. Tong Chen led the classification of *Deployment and Inference* approaches. Ruiqi Zheng contributed to writing the *Deployment and Inference* section. Jing Long contributed to writing the *Training and Updating* section. Xin Xia contributed to writing the *Deployment and Inference* section. Yuhui Shi contributed to the

revision and quality improvement of the manuscript. Chengqi Zhang contributed to the revision and quality improvement of the manuscript.

Funding The Australian Research Council partially supports this work under the streams of Future Fellowship (Grant No. FT210100624), Discovery Early Career Researcher Award (Grants No. DE230101033), the Discovery Project (Grant No. DP240101108 and DP240101814), and the Linkage Projects (Grant No. LP230200892 and LP240200546).

Data Availability This is a survey article and does not involve any original experimental data or materials.

Conflict of interest The authors declare that they have no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Zhou G, Zhu X, Song C, Fan Y, Zhu H, Ma X, Yan Y, Jin J, Li H, Gai K (2018) Deep interest network for click-through rate prediction. In: SIGKDD, pp 1059–1068
2. Covington P, Adams J, Sargin E (2016) Deep neural networks for youtube recommendations. In: RecSys, pp 191–198
3. Yin H, Sun Y, Cui B, Hu Z, Chen L (2013) Lcars: a location-content-aware recommender system. In: SIGKDD, pp 221–229
4. Yin H, Cui B, Zhou X, Wang W, Huang Z, Sadiq S (2016) Joint modeling of user check-in behaviors for real-time point-of-interest recommendation. ACM Trans Inf Syst 35(2):44
5. Yin H, Wang W, Wang H, Chen L, Zhou X (2017) Spatial-aware hierarchical collaborative deep learning for poi recommendation. IEEE Trans Knowl Data Eng 29(11):2537–2551
6. Yin H, Cui B, Huang Z, Wang W, Wu X, Zhou X (2015) Joint modeling of users' interests and mobility patterns for point-of-interest recommendation. In: MM, pp 819–822
7. Wang Q, Yin H, Chen T, Huang Z, Wang H, Zhao Y, Viet Hung NQ (2020) Next point-of-interest recommendation on resource-constrained mobile devices. In: WWW, pp 906–916
8. Guo H, Tang R, Ye Y, Li Z, He X (2017) Deepfm: a factorization-machine based neural network for ctr prediction. arXiv preprint [arXiv:1703.04247](https://arxiv.org/abs/1703.04247)
9. Ying R, He R, Chen K, Eksombatchai P, Hamilton W.L, Leskovec J (2018) Graph convolutional neural networks for web-scale recommender systems. In: SIGKDD, pp 974–983
10. Gong Y, Jiang Z, Feng Y, Hu B, Zhao K, Liu Q, Ou W (2020) EDGEREC: recommender system on edge in mobile Taobao. In: CIKM, pp 2477–2484
11. Gong X, Feng Q, Zhang Y, Qin J, Ding W, Li B, Jiang P, Gai K (2022) Real-time short video recommendation on mobile devices. In: CIKM, pp 3103–3112
12. Zhang H, Shen F, Liu W, He X, Luan H, Chua T-S (2016) Discrete collaborative filtering. In: SIGIR, pp 325–334

13. Zhang Y, Lian D, Yang G (2017) Discrete personalized ranking for fast collaborative filtering from implicit feedback. In: AAAI, pp 1669–1675
14. Liu S, Gao C, Chen Y, Jin D, Li Y (2021) Learnable embedding sizes for recommender systems
15. Qu Y, Chen T, Zhao X, Cui L, Zheng K, Yin H (2023) Continuous input embedding size search for recommender systems. In: SIGIR, pp 708–717
16. Shi H-JM, Mudigere D, Naumov M, Yang J (2020) Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In: SIGKDD, pp 165–175
17. Lian D, Wang H, Liu Z, Lian J, Chen E, Xie X (2020) Lightrec: a memory and search-efficient recommender system. In: WWW, pp 695–705
18. Xia X, Yin H, Yu J, Wang Q, Xu G, Nguyen QVH (2022) On-device next-item recommendation with self-supervised knowledge distillation. In: SIGIR, pp 546–555
19. Xia X, Yu J, Wang Q, Yang C, Hung NQV, Yin H (2023) Efficient on-device session-based recommendation. *ACM Trans Inf Syst* 41(4):1–24
20. Liu H, Zhao X, Wang C, Liu X, Tang J (2020) Automated embedding size search in deep recommender systems. In: SIGIR, pp 2307–2316
21. Chen T, Yin H, Zheng Y, Huang Z, Wang Y, Wang M (2021) Learning elastic embeddings for customizing on-device recommenders. In: SIGKDD, pp 138–147
22. Kang W.-C, Cheng D.Z, Yao T, Yi X, Chen T, Hong L, Chi E.H (2021) Learning to embed categorical features without embedding tables for recommendation. In: SIGKDD, pp 840–850
23. Xia X, Yu J, Xu G, Yin H (2023) Towards communication-efficient model updating for on-device session-based recommendation. In: CIKM, pp 2795–2804
24. Sun Z, Xu Y, Liu Y, He W, Jiang Y, Wu F, Cui L (2022) A survey on federated recommendation systems. *arXiv preprint arXiv:2301.00767*
25. Yang L, Tan B, Zheng V.W, Chen K, Yang Q (2020) Federated recommendation systems. *Federated Learning: Privacy and Incentive*, pp 225–239
26. Alamgir Z, Khan FK, Karim S (2022) Federated recommenders: methods, challenges and future. *Clust Comput* 25(6):4075–4096
27. Long J, Chen T, Nguyen QVH, Yin H (2023) Decentralized collaborative learning framework for next poi recommendation. *ACM Trans Inf Syst* 41(3):1–25
28. Qu L, Tang N, Zheng R, Nguyen QVH, Huang Z, Shi Y, Yin H (2023) Semi-decentralized federated ego graph learning for recommendation. In: WWW, pp 339–348
29. Long J, Chen T, Hung NQV, Xu G, Zheng K, Yin H (2023) Model-agnostic decentralized collaborative learning for on-device poi recommendation. In: SIGIR, pp 423–432
30. Wang Q, Yin H, Chen T, Yu J, Zhou A, Zhang X (2021) Fast-adapting and privacy-preserving federated recommender system. *VLDB J* 31(5):877–896
31. Yao J, Wang F, Jia K, Han B, Zhou J, Yang H (2021) Device-cloud collaborative learning for recommendation. In: SIGKDD, pp 3865–3874
32. Yan Y, Niu C, Gu R, Wu F, Tang S, Hua L, Lyu C, Chen G (2022) On-device learning for model personalization with large-scale cloud-coordinated domain adaption. In: SIGKDD, pp 2180–2190
33. Yuan W, Yang C, Nguyen QVH, Cui L, He T, Yin H (2023) Interaction-level membership inference attack against federated recommender systems. In: WWW, pp 1053–1062
34. Chai D, Wang L, Chen K, Yang Q (2022) Efficient federated matrix factorization against inference attacks. *ACM Trans Intel Syst Technol* 13(4):1–20
35. Yuan W, Yin H, Wu F, Zhang S, He T, Wang H (2023) Federated unlearning for on-device recommendation. In: WSDM, pp 393–401
36. Zhang S, Yuan W, Yin H (2023) Comprehensive privacy analysis on federated recommender system against attribute inference attacks. *IEEE Trans Knowl Data Eng* 36:987
37. Zhang S, Yin H, Chen T, Huang Z, Nguyen Q.V.H, Cui L (2022) Pipattack: poisoning federated recommender systems for manipulating item promotion. In: WSDM, pp 1415–1423
38. Wu C, Wu F, Qi T, Huang Y, Xie X (2022) Fedattack: effective and covert poisoning attack on federated recommendation via hard sampling. In: SIGKDD, pp 4164–4172
39. Yuan W, Nguyen Q.V.H, He T, Chen L, Yin H (2023) Manipulating federated recommender systems: poisoning with synthetic users and its countermeasures. *SIGIR*, pp 1690–1699
40. Gao C, Wang X, He X, Li Y (2022) Graph neural networks for recommender system. In: WSDM, pp 1623–1625
41. Wu S, Sun F, Zhang W, Xie X, Cui B (2022) Graph neural networks in recommender systems: a survey. *ACM Comput Surv* 55(5):37
42. Zheng R, Qu L, Cui B, Shi Y, Yin H (2023) Automl for deep recommender systems: a survey. *ACM Trans Inf Syst* 41(4):38
43. Tang R, Chen B, Wang Y, Guo H, Liu Y, Fan W, Zhao X (2023) Automl for deep recommender systems: fundamentals and advances. In: WSDM, pp 1264–1267
44. Yu J, Yin H, Xia X, Chen T, Li J, Huang Z (2022) Self-supervised learning for recommender systems: a survey. *arXiv preprint arXiv:2203.15876*
45. Li S, Guo H, Tang X, Tang R, Hou L, Li R, Zhang R (2023) Embedding compression in recommender systems: a survey. *ACM Comput Surv* 56(5):21
46. Zhang Q, Yang P, Yu J, Wang H, He X, Yiu S.-M, Yin H (2024) A survey on point-of-interest recommendation: models, architectures, and security. *arXiv preprint arXiv:2410.02191*
47. Himeur Y, Sohail SS, Bensaali F, Amira A, Alazab M (2022) Latest trends of security and privacy in recommender systems: a comprehensive review and future perspectives. *Comput Secur* 118:102746
48. Rafi TH, Noor FA, Hussain T, Chae D-K (2024) Fairness and privacy preserving in federated learning: a survey. *Inf Fusion* 105:102198
49. Wang S, Zhang X, Wang Y, Ricci F (2024) Trustworthy recommender systems. *ACM Trans Intel Syst Technol* 15(4):1–20
50. Dhar S, Guo J, Liu JJ, Tripathi S, Kurup U, Shah M (2021) A survey of on-device machine learning: an algorithms and learning theory perspective. *ACM Trans Internet Things* 2(3):49
51. Zhou Q, Qu Z, Guo S, Luo B, Guo J, Xu Z, Akerkar R (2021) On-device learning systems for edge intelligence: a software and hardware synergy perspective. *IEEE Internet Things J* 8(15):11916–11934
52. Asad M, Shaikat S, Javanmardi E, Nakazato J, Tsukada M (2023) A comprehensive survey on privacy-preserving techniques in federated recommendation systems. *Appl Sci* 13(10):6201
53. Harasic M, Keese F-S, Mattern D, Paschke A (2024) Recent advances and future challenges in federated recommender systems. *Int J Data Sci Anal* 17(4):337–357
54. Yang E, Xiong Y, Yuan W, Pan W, Yang Q, Ming Z (2025) A survey on cross-user federated recommendation. *Sci China Inf Sci* 68(4):1–26
55. Ali W, Zhou X, Shao J (2025) Privacy-preserved and responsible recommenders: from conventional defense to federated learning and blockchain. *ACM Comput. Surv.* 57(5) <https://doi.org/10.1145/3708982>
56. Koren Y, Bell R, Volinsky C (2009) Matrix factorization techniques for recommender systems. *Computer* 42(8):30–37

57. Lei X, Qian X, Zhao G (2016) Rating prediction based on social sentiment from textual reviews. *IEEE Trans Multimed* 18(9):1910–1921
58. Hidasi B, Karatzoglou A (2018) Recurrent neural networks with top-k gains for session-based recommendations. In: *CIKM*, pp 843–852
59. Xu Z, Li D, Zhao W, Shen X, Huang T, Li X, Li P (2021) Agile and accurate CTR prediction model training for massive-scale online advertising systems. In: *SIGMOD*, pp 2404–2409
60. Sun Y, Yuan F, Yang M, Wei G, Zhao Z, Liu D (2020) A generic network compression framework for sequential recommender systems. In: *SIGIR*
61. Yuan F, He X, Karatzoglou A, Zhang L (2020) Parameter-efficient transfer from sequential behaviors for user modeling and recommendation. In: *SIGIR*, pp 1469–1478
62. Wang J, Yuan F, Chen J, Wu Q, Yang M, Sun Y, Zhang G (2021) Stackrec: efficient training of very deep sequential recommender models by iterative stacking. In: *SIGIR*
63. Liu Z, Cheng M, Li Z, Liu Q, Chen E (2022) One person, one model-learning compound router for sequential recommendation. In: *ICDM*, pp 289–298
64. Chen L, Yuan F, Yang J, Ao X, Li C, Yang M (2021) A user-adaptive layer selection framework for very deep sequential recommender models. In: *AAAI*, pp 3984–3991
65. Yan B, Wang P, Zhang K, Lin W, Lee K.-C, Xu J, Zheng B (2021) Learning effective and efficient embedding via an adaptively-masked twins-based layer. *arXiv preprint arXiv:2108.11513*
66. Zhang Z, Wang Q, Ruan L, Si L (2014) Preference preserving hashing for efficient recommendation. In: *SIGIR*, pp 183–192
67. Joglekar MR, Li C, Chen M, Xu T, Wang X, Adams JK, Khaitan P, Liu J, Le QV (2020) Neural input search for large scale recommendation models. In: *SIGKDD*, pp 2387–2397
68. Liu X, He J, Deng C, Lang B (2014) Collaborative hashing. In: *CVPR*, pp 2139–2146
69. Wu L, He X, Wang X, Zhang K, Wang M (2022) A survey on accuracy-oriented neural recommendation: from collaborative filtering to information-rich recommendation. *IEEE Trans Knowl Data Eng* 35(5):4425–4445
70. Lian D, Liu R, Ge Y, Zheng K, Xie X, Cao L (2017) Discrete content-aware matrix factorization. In: *SIGKDD*, pp 325–334
71. Kang W.-C, McAuley J (2019) Candidate generation with binary codes for large-scale top-n recommendation. In: *CIKM*, pp 1523–1532
72. Courbariaux M, Bengio Y, David J-P (2015) Binaryconnect: training deep neural networks with binary weights during propagations. In: *NeurIPS*, pp 3123–3131
73. Tan Q, Liu N, Zhao X, Yang H, Zhou J, Hu X (2020) Learning to hash with graph neural networks for recommender systems. In: *WWW*, pp 1988–1998
74. Guan H, Malevich A, Yang J, Park J, Yuen H (2019) Post-training 4-bit quantization on embedding tables. *arXiv preprint arXiv:1911.02079*
75. Li S, Guo H, Hou L, Zhang W, Tang X, Tang R, Zhang R, Li R (2023) Adaptive low-precision training for embeddings in click-through rate prediction. In: *AAAI*, pp 4435–4443
76. Lyu F, Tang X, Zhu H, Guo H, Zhang Y, Tang R, Liu X (2022) Optembed: learning optimal embedding table for click-through rate prediction. In: *CIKM*, pp 1399–1409
77. Park Y.-J, Tuzhilin A (2008) The long tail of recommender systems and how to leverage it. In: *RecSys*, pp 11–18
78. Ginart A, Naumov M, Mudigere D, Yang J, Zou J (2019) Mixed dimension embeddings with application to memory-efficient recommendation systems. In: *ISIT*, pp 2786–2791
79. Pham H, Guan M, Zoph B, Le Q, Dean J (2018) Efficient neural architecture search via parameters sharing. In: *ICML*, pp 4095–4104
80. Cheng W, Shen Y, Huang L (2020) Differentiable neural input search for recommender systems. *arXiv*
81. Zhao X, Wang C, Chen M, Zheng X, Liu X, Tang J (2020) Autoemb: automated embedding dimensionality search in streaming recommendations. *arXiv preprint arXiv:2002.11252*
82. Zhao X, Liu H, Liu H, Tang J, Guo W, Shi J, Wang S, Gao H, Long B (2021) Autodim: field-aware embedding dimension search in recommender systems. In: *WWW*, pp 3015–3022
83. Deng W, Pan J, Zhou T, Kong D, Flores A, Lin G (2021) Deep-light: deep lightweight feature interactions for accelerating CTR predictions in ad serving. In: *WSDM*, pp 922–930
84. Qu L, Ye Y, Tang N, Zhang L, Shi Y, Yin H (2022) Single-shot embedding dimension search in recommender system. In: *SIGIR*, pp 513–522
85. Pansare N, Katukuri J, Arora A, Cipollone F, Shaik R, Tokgozoglu N, Venkataraman C (2022) Learning compressed embeddings for on-device inference. *Proc Mach Learn Syst* 4:382–397
86. Yan B, Wang P, Liu J, Lin W, Lee K.-C, Xu J, Zheng B (2021) Binary code based hash embedding for web-scale applications. In: *CIKM*, pp 3563–3567
87. Yin C, Acun B, Wu C.-J, Liu X (2021) Tt-rec: Tensor train compression for deep learning recommendation models. *Proc Mach Learn Syst* 3:448–462
88. Ammad-Ud-Din M, Ivannikova E, Khan S.A, Oyomno W, Fu Q, Tan K.E, Flanagan A (2019) Federated collaborative filtering for privacy-preserving personalized recommendation system. *arXiv preprint arXiv:1901.09888*
89. Chai D, Wang L, Chen K, Yang Q (2020) Secure federated matrix factorization. *IEEE Intel Syst* 36(5):11–20
90. Chen C, Li L, Wu B, Hong C, Wang L, Zhou J (2020) Secure social recommendation based on secret sharing. In: *ECAI*, pp 506–512
91. Muhammad K, Wang Q, O'Reilly-Morgan D, Tragos E, Smyth B, Hurley N, al(2020) Fedfast: going beyond average for faster training of federated recommender systems. In: *SIGKDD*, pp 1234–1242
92. Qi T, Wu F, Wu C, Huang Y, Xie X (2020) Privacy-preserving news recommendation model learning. *arXiv preprint arXiv:2003.09592*
93. Lin Y, Ren P, Chen Z, Ren Z, Yu D, Ma J, Rijke MD, Cheng X (2020) Meta matrix factorization for federated rating predictions. In: *SIGIR*, pp 981–990
94. Anelli V.W, Deldjoo Y, Di Noia T, Ferrara A, Narducci F (2021) Federank: user controlled feedback with federated recommender systems. In: *ECIR*, pp 32–47
95. Cui J, Chen C, Lyu L, Yang C, Li W (2021) Exploiting data sparsity in secure cross-platform social recommendation. In: *NeurIPS*, pp 10524–10534
96. Minto L, Haller M, Livshits B, Haddadi H (2021) Stronger privacy for federated collaborative filtering with implicit feedback. In: *RecSys*, pp 342–350
97. Lin G, Liang F, Pan W, Ming Z (2020) Fedrec: federated recommendation with explicit feedback. *IEEE Intel Syst* 36(5):21–30
98. Liang F, Pan W, Ming Z (2021) Fedrec++: lossless federated recommendation with explicit feedback. In: *AAAI*, pp 4224–4231
99. Khan F.K, Flanagan A, Tan K.E, Alamgir Z, Ammad-ud-din M (2021) A payload optimization method for federated recommender systems. In: *RecSys*, pp 432–442
100. Liu S, Xu S, Yu W, Fu Z, Zhang Y, Marian A (2021) Fedct: federated collaborative transfer for recommendation. In: *SIGIR*, pp 716–725

101. Wu J, Liu Q, Huang Z, Ning Y, Wang H, Chen E, Yi J, Zhou B (2021) Hierarchical personalized federated learning for user modeling. In: WWW, pp 957–968
102. Yang E, Huang Y, Liang F, Pan W, Ming Z (2021) FCMF: federated collective matrix factorization for heterogeneous collaborative filtering. *Knowl Based Syst* 220:106946
103. Chen C, Wu H, Su J, Lyu L, Zheng X, Wang L (2022) Differential private knowledge transfer for privacy-preserving cross-domain recommendation. In: WWW, pp 1455–1465
104. Liu S, Ge Y, Xu S, Zhang Y, Marian A (2022) Fairness-aware federated matrix factorization. In: RecSys, pp 168–178
105. Perifanis V, Efrimidis PS (2022) Federated neural collaborative filtering. *Know Based Syst* 242:108441
106. Wu C, Wu F, Lyu L, Qi T, Huang Y, Xie X (2022) A federated graph neural network framework for privacy-preserving personalization. *Nat Commun* 13(1):3091
107. Liu Z, Yang L, Fan Z, Peng H, Yu PS (2022) Federated social recommendation with graph neural network. *ACM Trans Intel Syst Technol* 13(4):1–24
108. Luo S, Xiao Y, Song L (2022) Personalized federated recommendation via joint representation learning, user clustering, and model adaptation. In: CIKM, pp 4289–4293
109. Wu C, Wu F, Lyu L, Huang Y, Xie X (2022) FedCTR: Federated native ad CTR prediction with cross-platform user behavior data. *ACM Trans Intel Syst Technol* 13(4):1–19
110. Meihan W, Li L, Tao C, Rigall E, Xiaodong W, Cheng-Zhong X (2022) FedCDR: federated cross-domain recommendation for privacy-preserving rating prediction. In: CIKM, pp 2179–2188
111. Lin Z, Pan W, Yang Q, Ming Z (2022) A generic federated recommendation framework via fake marks and secret sharing. *ACM Trans Inf Syst* 41(2):37
112. Huang W, Liu J, Li T, Huang T, Ji S, Wan J (2023) FedDSR: daily schedule recommendation in a federated deep reinforcement learning framework. *IEEE Trans Knowl Data Eng* 35(4):3912–3924
113. Liu R, Cao Y, Wang Y, Lyu L, Chen Y, Chen H (2023) Privaterec: differentially private model training and online serving for federated news recommendation. In: SIGKDD, pp 4539–4548
114. Liu W, Chen C, Liao X, Hu M, Yin J, Tan Y, Zheng L (2023) Federated probabilistic preference distribution modelling with compactness co-clustering for privacy-preserving multi-domain recommendation. In: IJCAI, pp 2206–2214
115. Cao Z, Liang Z, Wu B, Zhang S, Li H, Wen O, al (2023) Privacy matters: vertical federated linear contextual bandits for privacy protected recommendation. In: KDD, pp 154–166
116. Imran M, Yin H, Chen T, Nguyen QVH, Zhou A, Zheng K (2023) ReFRS: resource-efficient federated recommender system for dynamic and diversified user preferences. *ACM Trans Inf Syst* 41(3):30
117. Zhang C, Long G, Zhou T, Yan P, Zhang Z, Zhang C, Yang B (2023) Dual personalization on federated recommendation. *arXiv preprint arXiv:2301.08143*
118. Zhang H, Luo F, Wu J, He X, Li Y (2023) Lightfr: lightweight federated recommendation with privacy-preserving matrix factorization. *ACM Trans Inf Syst* 41(4):1–28
119. Ye Z, Zhang X, Chen X, Xiong H, Yu D (2024) Adaptive clustering based personalized federated learning framework for next poi recommendation with location noise. *IEEE Trans Knowl Data Eng* 36(5):1843–1856. <https://doi.org/10.1109/TKDE.2023.3312511>
120. Li Z, Wu X, Pan W, Ding Y, Wu Z, Tan S, Xu Q, Yang Q, Ming Z (2024) Fedcore: federated learning for cross-organization recommendation ecosystem. *IEEE Trans Knowl Data Eng* 36(8):3817–3831. <https://doi.org/10.1109/TKDE.2024.3363505>
121. Zhang C, Long G, Zhou T, Zhang Z, Yan P, Yang B (2024) Gpfedrec: graph-guided personalization for federated recommendation. in: proceedings of the 30th acm sigkdd conference on Knowledge Discovery and Data Mining. KDD '24, pp 4131–4142. Association for Computing Machinery New York, NY, USA. <https://doi.org/10.1145/3637528.3671702>
122. Li Z, Long G, Zhou T (2024) Federated recommendation with additive personalization. In: The Twelfth international conference on learning representations. <https://openreview.net/forum?id=xkXdE81mOK>
123. He X, Liu S, Keung J, He J (2024) Co-clustering for federated recommender system. In: Proceedings of the ACM Web Conference 2024. WWW '24, pp 3821–3832. Association for Computing Machinery New York, NY, USA. <https://doi.org/10.1145/3589334.3645626>
124. Yan B, Cao Y, Wang H, Yang W, Du J, Shi C (2024) Federated heterogeneous graph neural network for privacy-preserving recommendation. In: Proceedings of the ACM Web Conference 2024. WWW '24, pp 3919–3929. Association for Computing Machinery New York, NY, USA. <https://doi.org/10.1145/3589334.3645693>
125. Di Y, Shi H, Wang X, Ma R, Liu Y (2025) Federated recommender system based on diffusion augmentation and guided denoising. *ACM Trans. Inf. Syst.* 43(2) <https://doi.org/10.1145/3688570>
126. Yuan W, Yang C, Qu L, Hung Nguyen QV, Ye G, Yin H (2025) Ptf-fsr: a parameter transmission-free federated sequential recommender system. *ACM Trans Inf Syst* 43(2):1–24. <https://doi.org/10.1145/3708344>
127. Li Z, Long G, Zhou T, Jiang J, Zhang C (2025) Personalized federated collaborative filtering: a variational autoencoder approach. *Proc AAAI Conf Artif Intel* 39:18602–18610
128. Lin Z, Pan W, Ming Z (2021) Fr-FMSS: federated recommendation via fake marks and secret sharing. In: RecSys, pp 668–673
129. McMahan B, Moore E, Ramage D, Hampson S, Arcas BAY (2017) Communication-efficient learning of deep networks from decentralized data. In: AISTATS, pp 1273–1282
130. Wang S, Ji M (2022) A unified analysis of federated learning with arbitrary client participation. In: NeurIPS, pp 19124–19137
131. He X, Chua T-S (2017) Neural factorization machines for sparse predictive analytics. In: SIGIR, pp 355–364
132. Ma H, Zhou D, Liu C, Lyu MR, King I (2011) Recommender systems with social regularization. In: WSDM, pp 287–296
133. He X, Liao L, Zhang H, Nie L, Hu X, Chua T-S (2017) Neural collaborative filtering. In: WWW, pp 173–182
134. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2017) Graph attention networks. *arXiv preprint arXiv:1710.10903*
135. Abbasi-yadkori Y, Pál D, Szepesvári C (2011) Improved algorithms for linear stochastic bandits. In: NeurIPS, pp. 2312–2320
136. Agrawal S, Goyal N (2013) Thompson sampling for contextual bandits with linear payoffs. In: ICML, pp 127–135
137. Melis L, Song C, De Cristofaro E, Shmatikov V (2019) Exploiting unintended feature leakage in collaborative learning. In: SP, pp 691–706
138. Bonawitz K, Ivanov V, Kreuter B, Marcedone A, McMahan HB, Patel S, et al (2017) Practical secure aggregation for privacy-preserving machine learning. In: SIGSAC, pp 1175–1191
139. Reddi S, Charles Z, Zaheer M, Garrett Z, Rush K, Konečný J, Kumar S, McMahan HB (2020) Adaptive federated optimization. *arXiv preprint arXiv:2003.00295*
140. Ji S, Pan S, Long G, Li X, Jiang J, Huang Z (2019) Learning private neural language modeling with attentive aggregation. In: IJCNN, pp 1–8
141. Fallah A, Mokhtari A, Ozdaglar A (2020) Personalized federated learning with theoretical guarantees: a model-agnostic meta-learning approach. In: NeurIPS, pp 3557–3568

142. T Dinh C, Tran N, Nguyen J (2020) Personalized federated learning with Moreau envelopes. In: *NeurIPS*, pp 21394–21405
143. Wang Z, Liu X, Chang S, Zhou J, Qi G-J, Huang TS (2015) Decentralized recommender systems. *arXiv preprint arXiv:1503.01647*
144. Kermarrec A-M, Leroy V, Moin A, Thraves C (2010) Application of random walks to decentralized recommender systems. In: *OPODIS*, pp 48–63
145. Chen C, Liu Z, Zhao P, Zhou J, Li X (2018) Privacy preserving point-of-interest recommendation using decentralized matrix factorization. In: *AAAI*, pp 257–264
146. Ling Q, Xu Y, Yin W, Wen Z (2012) Decentralized low-rank matrix completion. In: *ICASSP*, pp 2925–2928
147. Hegedűs I, Danner G, Jelasity M (2019) Decentralized recommendation based on matrix factorization: A comparison of gossip and federated learning. In: *ECML PKDD*, pp 317–332
148. Defiebre D, Sacharidis D, Germanakos P (2020) A decentralized recommendation engine in the social internet of things. In: *UMAP*, pp 77–82
149. Defiebre D, Sacharidis D, Germanakos P (2022) A human-centered decentralized architecture and recommendation engine in Siot. *User Model User-Adap Inter* 32(3):297–353
150. Yang X, Luo Y, Fu S, Xu M, Chen Y (2022) Dpmf: decentralized probabilistic matrix factorization for privacy-preserving recommendation. *Appl Sci* 12(21):11118
151. An J, Li G, Jiang W (2024) Nrdl: decentralized user preference learning for privacy-preserving next poi recommendation. *Expert Syst Appl* 239:122421
152. Zheng X, Wang Z, Chen C, Qian J, Yang Y (2023) Decentralized graph neural network for privacy-preserving recommendation. In: *CIKM*, pp 3494–3504
153. Beierle F and Eichinger T (2019) Collaborating with users in proximity for decentralized mobile recommender systems. In: *SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI*, pp 1192–1197
154. Zhu L, Liu Z, Han S (2019) Deep leakage from gradients. In: *NeurIPS*, pp 14774–14784
155. Yin H, Mallya A, Vahdat A, Alvarez J.M, Kautz J, Molchanov P (2021) See through gradients: image batch recovery via gradinversion. In: *CVPR*, pp 16337–16346
156. Nichol A, Achiam J, Schulman J (2018) On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*
157. Mairitha N, Mairitha T, Inoue S (2020) Improving activity data collection with on-device personalization using fine-tuning. In: *UbiComp/ISWC*, pp 255–260
158. Wu C, Wu F, Cao Y, Huang Y, Xie X (2021) FedGNN: federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925*
159. Zhang S, Yin H, Chen T, Huang Z, Cui L, Zhang X (2021) Graph embedding for recommendation against attribute inference attacks. In: *WWW*, pp 3002–3014
160. Beigi G, Mosallanezhad A, Guo R, Alvari H, Nou A, Liu H (2020) Privacy-aware recommendation with private-attribute protection using adversarial learning. In: *WSDM*, pp 34–42
161. Harding EL, Vanto JJ, Clark R, Hannah Ji L, Ainsworth SC (2019) Understanding the scope and impact of the California consumer privacy act of 2018. *J Data Prot Privacy* 2(3):234–253
162. Dwork C, McSherry F, Nissim K, Smith A. Calibrating noise to sensitivity in private data analysis. In: *TCC*, pp 265–284 (2006)
163. Dwork C (2008) Differential privacy: a survey of results. In: *TAMC*, pp 1–19
164. Gentry C (2009) Fully homomorphic encryption using ideal lattices. In: *STOC*, pp 169–178
165. Shamir A (1979) How to share a secret. *Commun ACM* 22(11):612–613
166. Luo S, Xiao Y, Zhang X, Liu Y, Ding W, Song L (2023) Perfed-rec++: enhancing personalized federated recommendation with self-supervised pre-training. *arXiv preprint arXiv:2305.06622*
167. Perifanis V, Drosatos G, Stamatiatos G, Efraimidis PS (2023) Fedpoirec: privacy-preserving federated poi recommendation with social influence. *Inf Sci* 623:767–790
168. Li D, Chen C, Lv Q, Shang L, Zhao Y, Lu T, Gu N (2016) An algorithm for efficient privacy-preserving item-based collaborative filtering. *Futur Gener Comput Syst* 55:311–320
169. Ying S (2020) Shared mf: a privacy-preserving recommendation system. *arXiv preprint arXiv:2008.07759*
170. Angel S, Chen H, Laine K, Setty S (2018) PIR with compressed queries and amortized query processing. In: *SP*, pp 962–979
171. Yuan W, Yang C, Qu L, Nguyen Q.V.H, Li J, Yin H (2023) Hide your model: a parameter transmission-free federated recommender system. *arXiv preprint arXiv:2311.14968*
172. Di Noia T, Malitesta D, Merra F.A (2020) TaAMR: targeted adversarial attack against multimedia recommender systems. In: *DSN-W*, pp 1–8
173. Yu Y, Liu Q, Wu L, Yu S.L, Zhang Z (2023) Untargeted attack against federated recommendation systems via poisonous item embeddings and the defense. In: *AAAI*, pp 4854–4863
174. Wu Z-W, Chen C-T, Huang S-H (2022) Poisoning attacks against knowledge graph-based recommendation systems using deep reinforcement learning. *Neural Comput Appl* 34:3097
175. Wu C, Lian D, Ge Y, Zhu Z, Chen E (2021) Triple adversarial learning for influence based poisoning attack in recommender systems. In: *SIGKDD*, pp 1830–1840
176. Lin C, Chen S, Li H, Xiao Y, Li L, Yang Q (2020) Attacking recommender systems with augmented user profiles. In: *CIKM*, pp 855–864
177. Fang M, Yang G, Gong N.Z, Liu J (2018) Poisoning attacks to graph-based recommender systems. In: *ACSAC*, pp 381–392
178. Christakopoulou K, Banerjee A (2019) Adversarial attacks on an oblivious recommender. In: *RecSys*, pp 322–330
179. Kapoor S, Kapoor V, Kumar R (2017) A review of attacks and its detection attributes on collaborative recommender systems. *Int J Adv Res Comput Sci*, 8(7)
180. Gunes I, Kaleli C, Bilge A, Polat H (2014) Shilling attacks against recommender systems: a comprehensive survey. *Artif Intel Rev* 42:767–799
181. Jiang Y, Zhou Y, Wu D, Li C, Wang Y (2020) On the detection of shilling attacks in federated collaborative filtering. In: *SRDS*, pp 185–194
182. Wu Z, Wu J, Cao J, Tao D (2012) Hysad: a semi-supervised hybrid shilling attack detector for trustworthy product recommendation. In: *SIGKDD*, pp 985–993
183. Xuan H, Stylianou A, Liu X, Pless R (2020) Hard negative examples are hard, but useful. In: *ECCV*, pp 126–142
184. Rong D, Ye S, Zhao R, Yuen H.N, Chen J, He Q (2022) Fedrecat: model poisoning attack to federated recommendation. In: *ICDE*, pp 2643–2655
185. Rong D, He Q, Chen J (2022) Poisoning deep learning based recommender model in federated learning scenarios. *arXiv preprint arXiv:2204.13594*
186. Yuan W, Yuan S, Yang C, Nguyen QVH, Yin H (2023) Manipulating visually-aware federated recommender systems and its countermeasures. *ACM Trans Inf Syst* 42(3):1–26
187. Yin M, Xu Y, Fang M, Gong N.Z (2024) Poisoning federated recommender systems with fake users. In: *Proceedings of the ACM Web Conference 2024. WWW '24*, pp 3555–3565. Association for Computing Machinery New York, NY, USA. <https://doi.org/10.1145/3589334.3645492>
188. Guerraoui R, Rouault S (2018) The hidden vulnerability of distributed learning in byzantium. In: *ICML*, pp 3521–3530

189. Yin D, Chen Y, Kannan R, Bartlett P (2018) Byzantine-robust distributed learning: towards optimal statistical rates. In: ICML, pp 5650–5659
190. Blanchard P, El Mhamdi E.M, Guerraoui R, Stainer J (2017) Machine learning with adversaries: byzantine tolerant gradient descent. In: NeurIPS, pp 119–129
191. Chen C, Zhang J, Tung AK, Kankanhalli M, Chen G (2020) Robust federated recommendation system. arXiv preprint [arXiv:2006.08259](https://arxiv.org/abs/2006.08259)
192. Liang R, Jiang Y, Zhu F, Cheng L, Liu H (2025) Defending federated recommender systems against untargeted attacks: a contribution-aware robust aggregation scheme. *ACM Trans Knowl Discov Data* 19(1):1–28
193. Desai A, Meisburger B, Liu Z, Shrivastava A (2023) Heterogeneous federated collaborative filtering using fair: federated averaging in random subspaces. arXiv preprint [arXiv:2311.01722](https://arxiv.org/abs/2311.01722)
194. Yuan W, Qu L, Cui L, Tong Y, Zhou X, Yin H (2023) Hetefedrec: federated recommender systems with model heterogeneity. arXiv preprint [arXiv:2307.12810](https://arxiv.org/abs/2307.12810)
195. Qu L, Yuan W, Zheng R, Cui L, Shi Y, Yin H (2024) Towards personalized privacy: user-governed data contribution for federated recommendation. arXiv preprint [arXiv:2401.17630](https://arxiv.org/abs/2401.17630)
196. Maeng K, Lu H, Melis L, Nguyen J, Rabbat M, Wu C-J (2022) Towards fair federated recommendation learning: characterizing the inter-dependence of system and data heterogeneity. In: *RecSys*, pp 156–167
197. Min B, Ross H, Sulem E, Veyseh APB, Nguyen TH, Sainz O, Agirre E, Heintz I, Roth D (2023) Recent advances in natural language processing via large pre-trained language models: a survey. *ACM Comput Surv* 56(2):1–40
198. Du Y, Liu Z, Li J, Zhao W.X (2022) A survey of vision-language pre-trained models. arXiv preprint [arXiv:2202.10936](https://arxiv.org/abs/2202.10936)
199. Fan W, Zhao Z, Li J, Liu Y, Mei X, Wang Y, Tang J, Li Q (2023) Recommender systems in the era of large language models (llms). arXiv preprint [arXiv:2307.02046](https://arxiv.org/abs/2307.02046)
200. He Z, Xie Z, Jha R, Steck H, Liang D, Feng Y, al (2023) Large language models as zero-shot conversational recommenders. In: *CIKM*, pp 720–730
201. Wei W, Ren X, Tang J, Wang Q, Su L, Cheng S, Wang J, Yin D, Huang C (2023) Llmrec: large language models with graph augmentation for recommendation. arXiv preprint [arXiv:2311.00423](https://arxiv.org/abs/2311.00423)
202. Cui Z, Ma J, Zhou C, Zhou J, Yang H (2022) M6-rec: generative pretrained language models are open-ended recommender systems. arXiv preprint [arXiv:2205.08084](https://arxiv.org/abs/2205.08084)
203. Liu J, Liu C, Lv R, Zhou K, Zhang Y (2023) Is chatgpt a good recommender? A preliminary study. arXiv preprint [arXiv:2304.10149](https://arxiv.org/abs/2304.10149)
204. Yuan W, Yang C, Ye G, Chen T, Nguyen Q.V.H (2024) Fellas: enhancing federated sequential recommendation with LLM as external services. arXiv preprint [arXiv:2410.04927](https://arxiv.org/abs/2410.04927)
205. Long J, Ye G, Chen T, Wang Y, Wang M, Yin H (2024) Diffusion-based cloud-edge-device collaborative learning for next poi recommendations. In: *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pp 2026–2036
206. Zhu J, Dai Q, Su L, Ma R, Liu J, Cai G, Xiao X, Zhang R (2022) Bars: Towards open benchmarking for recommender systems. In: *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval. SIGIR '22*, pp 2912–2923. Association for Computing Machinery New York, NY, USA. <https://doi.org/10.1145/3477495.3531723>
207. Yuan G, Yuan F, Li Y, Kong B, Li S, Chen L, Yang M, Yu C, Hu B, Li Z, Xu Y, Qi X (2022) Tenrec: a large-scale multipurpose benchmark dataset for recommender systems. In: *Proceedings of the 36th international conference on neural information processing systems. NIPS '22*. Curran Associates Inc. Red Hook, NY, USA
208. Tran HV, Chen T, Quoc Viet Hung N, Huang Z, Cui L, Yin H (2025) A thorough performance benchmarking on lightweight embedding-based recommender systems. *ACM Trans Inf Syst* 43(3):1–32. <https://doi.org/10.1145/3712589>